

TLS: Funzionalità

A horizontal yellow brushstroke with a textured, painterly appearance, spanning most of the width of the slide.

REMIND

- ❑ **Attacker** nel **canale di comunicazione** ("Network attacker")
 - ❑ **Osserva / Modifica / Fabbrica**
 - ❑ Qualsiasi messaggio in qualsiasi istante



TCP

No Secrecy
No Authentication
No Integrity

(ovviamente)



- ❑ **Attacker nel canale di comunicazione** ("Network attacker")
 - ❑ **Osserva / Modifica / Fabbrica**
 - ❑ **Qualsiasi messaggio in qualsiasi istante**

TLS:

Transport Layer Security

- ❑ Inizialmente si chiamava **SSL** (Secure Socket Layer)
- ❑ Varie versioni successive
- ❑ Tendenzialmente sinonimi

- ❑ Terminologia oggi adottata: TLS

- ❑ Vediamo:
 1. Funzionalità (senza chiederci come implementate)
 2. Implementazione

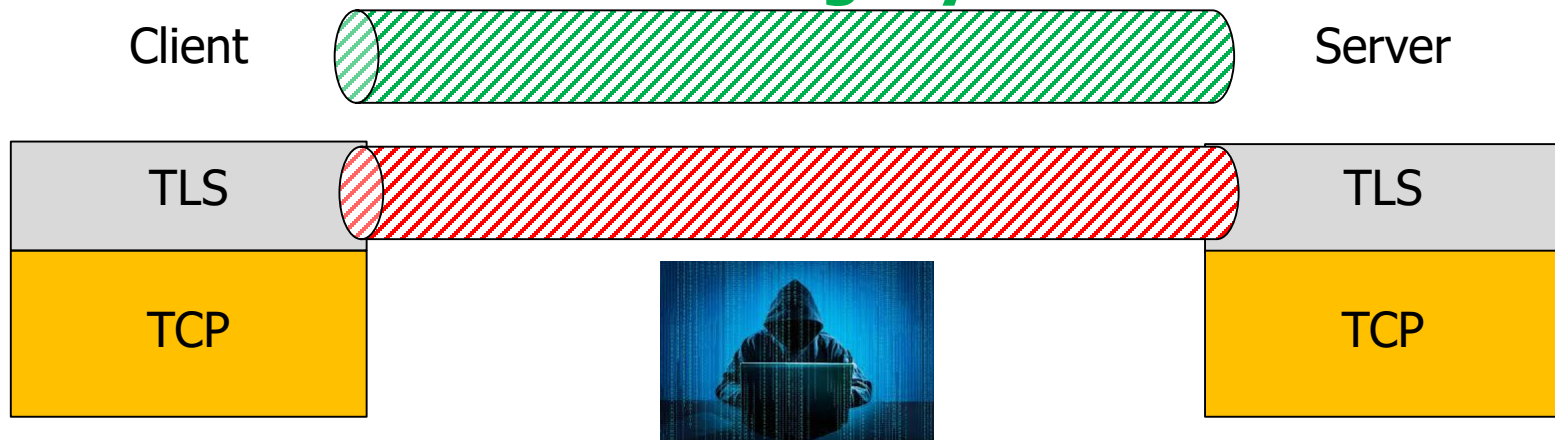
TLS: Proprietà

Utilizza tecniche **crittografiche** per “irrobustire” una connessione TCP

Secrecy

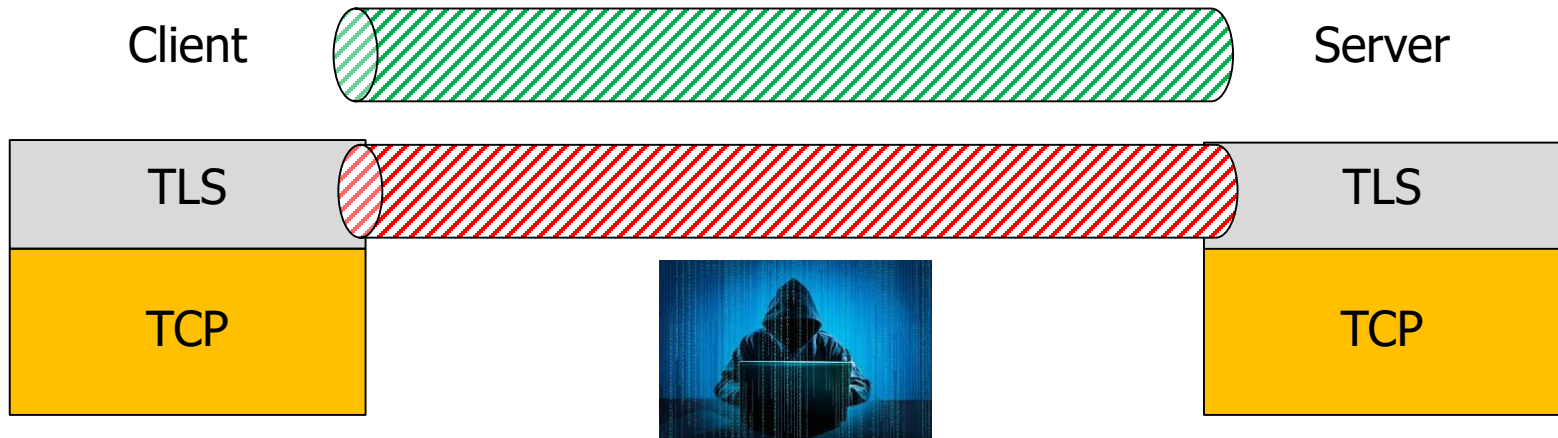
Authentication (Server DNS name)

Integrity



TLS: Utilizzo

- Un protocollo applicativo che sta sopra TCP può stare anche sopra TLS (occorre modificare i sorgenti del programma...)
 - HTTPS (TCP port 443)
 - SMTP over TLS (TCP port 465)
 - POP over TLS (TCP port 995)
 - ...



TLS: Configurazione (I)

C

☐ PersonalSet: **empty**



S

☐ PersonalSet: **KPRIV-S** **KPUB-S**

☐ KeySet: **CA-S**

☐ TrustSet: **CA-S**

Subject = **DNS-NAME**

KPUB = **KPUB-S**

Issuer = **CA-S**

☐ Server deve avere **coppia di chiavi e certificato**

☐ Client **non** occorre che ce le abbia

TLS: Configurazione (II)

C

- ❑ PersonalSet: empty
- ❑ KeySet: CA-S
- ❑ TrustSet: CA-S

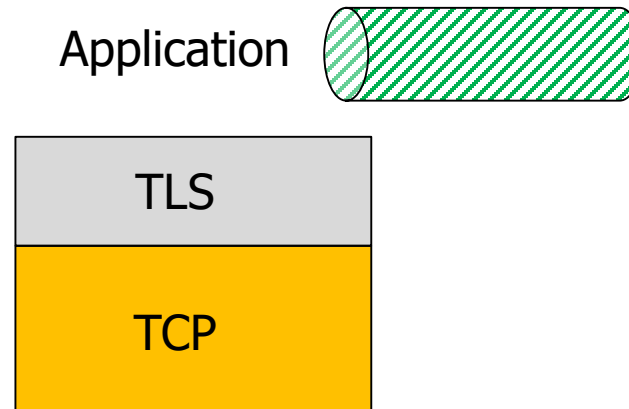
S

- ❑ PersonalSet: KPRIV-S KPUB-S
- ❑ KeySet: CA-S
- ❑ TrustSet: CA-S

Subject	=	DNS-NAME
KPUB	=	KPUB-S
Issuer	=	CA-S

- ❑ Certificato Server deve essere **utilizzabile** da Client

TLS: Secrecy



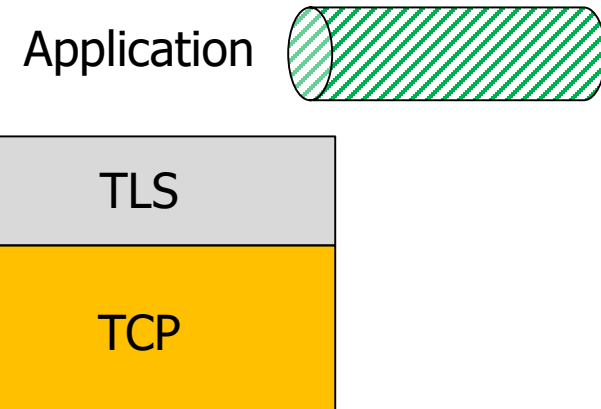
- ❑ Traffico applicativo crittato con crittografia a chiave **privata**
- ❑ Chiave **diversa** in **ogni** connessione
 - ❑ In teoria la stessa chiave si potrebbe usare per sempre
 - ❑ Prudent Engineering Practice: cambio chiave "frequente"

TLS: Integrity

- ❑ Dati alterati in transito



- ❑ Chiusura **automatica** connessione



- ❑ Un network attacker può impedirmi di lavorare (cade availability)
- ❑ Denial of Service (DoS): costo attacco molto inferiore al costo della difesa

TCP: No authentication

Non posso sapere se
all'altra estremità c'è:

1. Server con **IP-s**
2. Network Attacker

DNS

...		
DNS-NAME	A	IP-s
...		

Connect (DNS-Resolve (**DNS-NAME**) , ...)

Client



IP-s



TCP

TCP

TLS:

Server Authentication (I)

Sono certo che
all'altra estremità c'è:

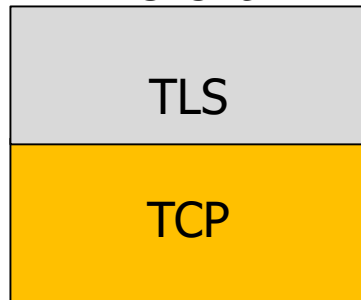
1. Server con **IP-s**
- ~~2. Network Attacker~~

DNS

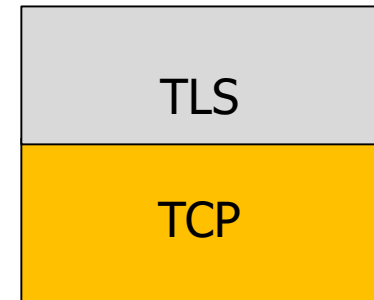
...		
DNS-NAME	A	IP-s
...		

Connect (DNS-Resolve (**DNS-NAME**), ...)

Client



IP-s



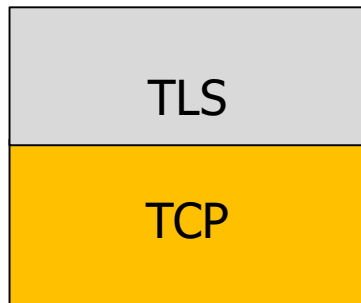
TLS:

Server Authentication (II)

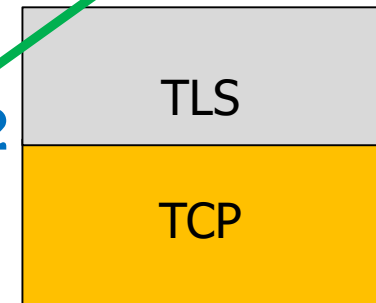
1. E' stato presentato un certificato per **DNS-NAME**
2. All'altra estremità c'è un processo che ha dimostrato di conoscere la KPRIV corretta

Connect (DNS-Resolve (**DNS-NAME**), ...)

Client



KPRIV-S



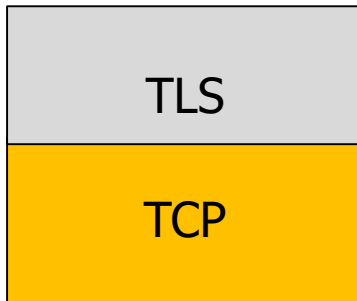
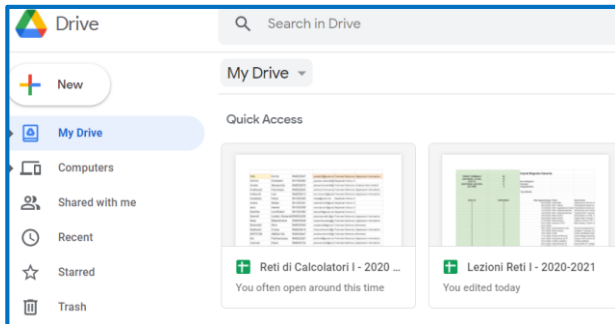
S = **DNS-NAME**

KPUB = **KPUB-S**

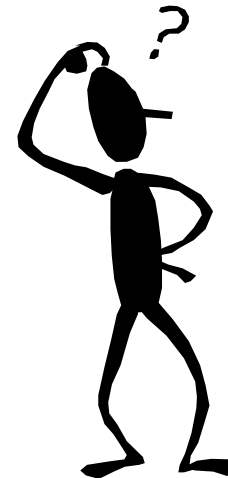
2

Esempio

<https://drive.google.com>

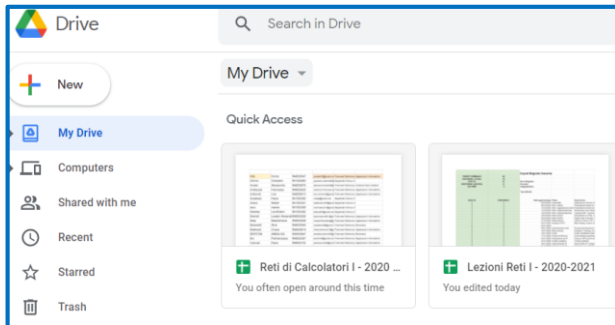


- ❑ Sono riuscito a collegarmi con `drive.google.com` **su** `https`
- ❑ Quali proprietà ho?



TLS: Secrecy

<https://drive.google.com>

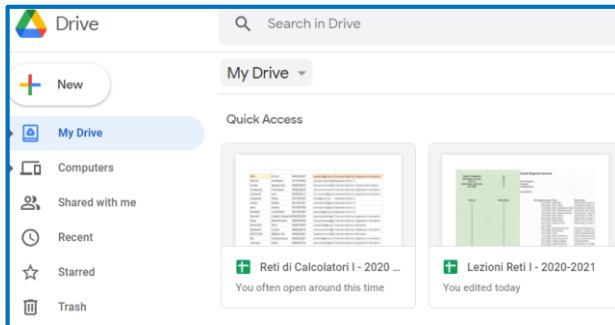


TLS

- ❑ Traffico applicativo crittato con crittografia a chiave **privata**
- ❑ Chiave **diversa** in **ogni** connessione
 - ❑ In teoria la stessa chiave si potrebbe usare per sempre
 - ❑ Prudent Engineering Practice: cambio chiave "frequente"

TLS: Integrity

<https://drive.google.com>



TLS

❑ Dati alterati in transito

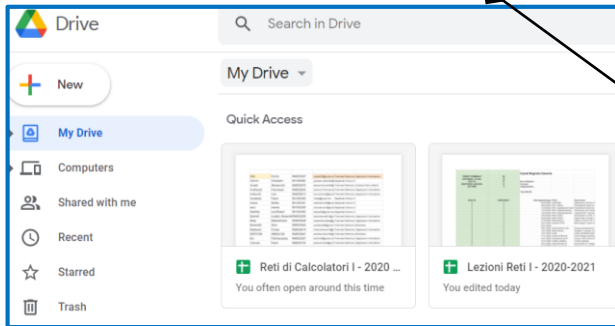


❑ Chiusura **automatica** connessione

TLS:

Server authentication (I)

<https://drive.google.com>



TLS



$S = \text{drive.google.com}$

$K_{PUB} = K_{PUB-S}$

K_{PRIV-S}

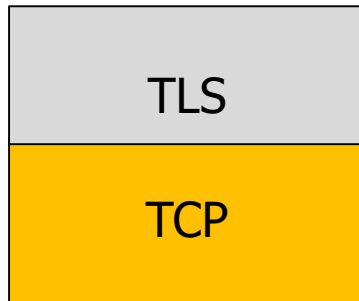
- ❑ E' stato presentato un certificato per drive.google.com
- ❑ Il server ha dimostrato di conoscere la K-PRIV corretta

TLS:

Server authentication (II)

Connect (DNS-Resolve (drive.google.com), ...)

Client

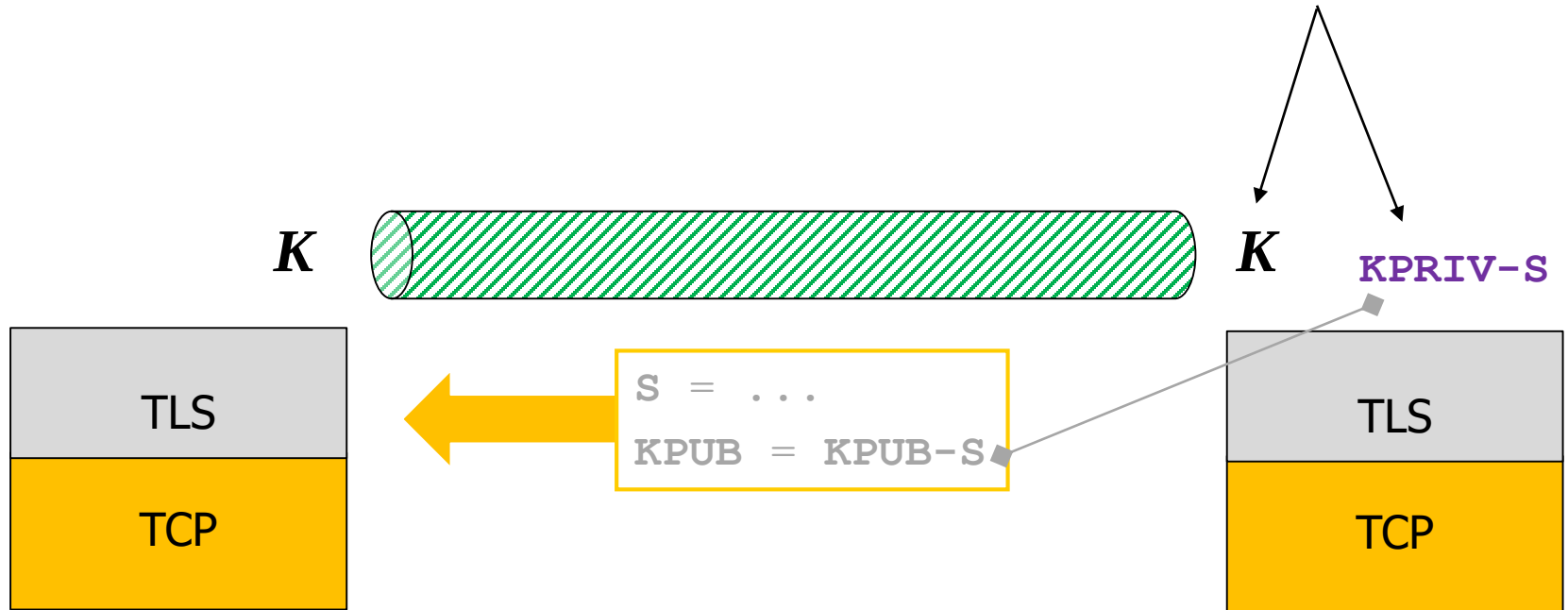


Non conosco K-priv
compagna della K-pub
di drive.google.com!!!

Nota bene

□ Sono **diverse**

□ Tecniche crittografiche **diverse**



Osservazione 1



- ❑ Network Attacker può **impedire** la connessione (Denial of Service)
- ❑ ...ma **non può** impersonare **DNS-NAME**

- ❑ Server authentication Garantita
- ❑ Availability NON Garantita

Osservazione 2

- ❑ E' stato presentato un certificato per `server-name`
- ❑ All'altra estremità c'è un processo che ha dimostrato di conoscere la KPRIV corretta



- ❑ **DNS-NAME dimostra** di conoscere un segreto
- ❑ ...senza che lo conosca **nessun altro** (!!!)



Osservazione 3



- ❑ Primo esempio di autenticazione **del Server**
- ❑ Client **non** si autentica

- ❑ POP / SMTP / HTTP Basic / HTTP FORM **su TCP:**
 - ❑ Client authentication
 - ❑ No Server authentication
- ❑ **TLS:**
 - ❑ No Client authentication
 - ❑ Server authentication

HTTPS (HTTP over TLS): Funzionalità



HTTPS (I)



- ❑ `https://...`
- ❑ Documento deve essere prelevato con **HTTP over TLS**

- ❑ HTTP port 80
- ❑ **HTTP over TLS** port 443
 - ❑ Secrecy
 - ❑ Server Authentication
 - ❑ Integrity

HTTPS (II)

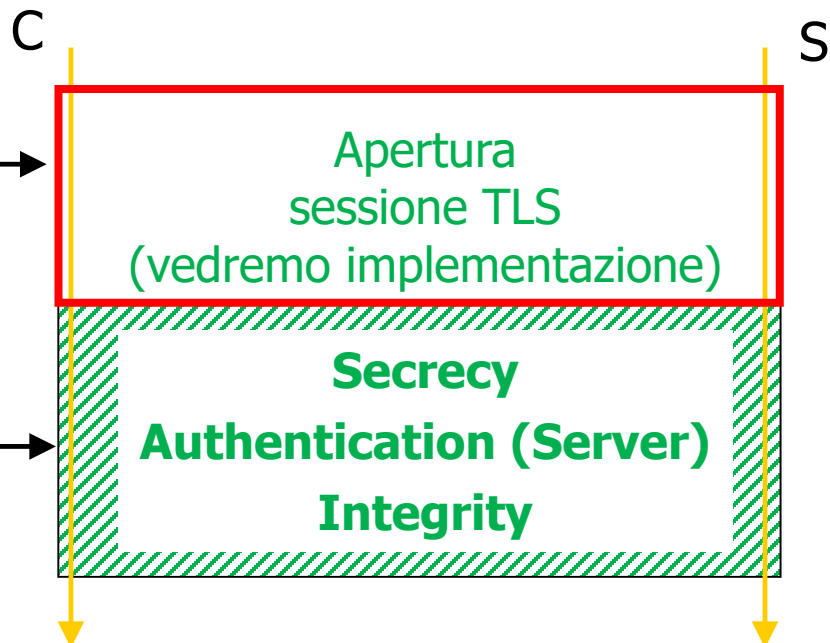
`https://DNS-NAME/someDoc.htm`

Browser:

1. Ottiene IP-N di **DNS-NAME**
2. `connect()` con porta **443** di IP-N

3. ... →

4. `send()/receive()`
messaggi HTTP →



HTTP

No Secrecy
No Authentication
No Integrity

(ovviamente)



- ❑ **Attacker nel canale di comunicazione** ("Network attacker")
 - ❑ **Osserva / Modifica / Fabbrica**
 - ❑ **Qualsiasi messaggio in qualsiasi istante**

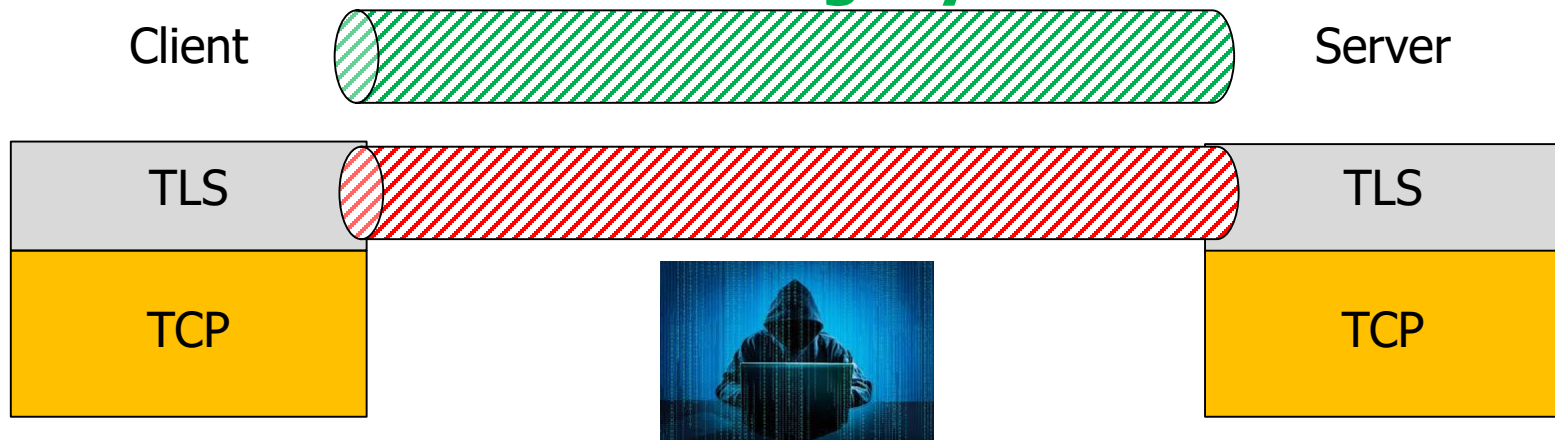
HTTPS (HTTP over TLS)

Utilizza tecniche **crittografiche** per "irrobustire" una connessione TCP

Secrecy

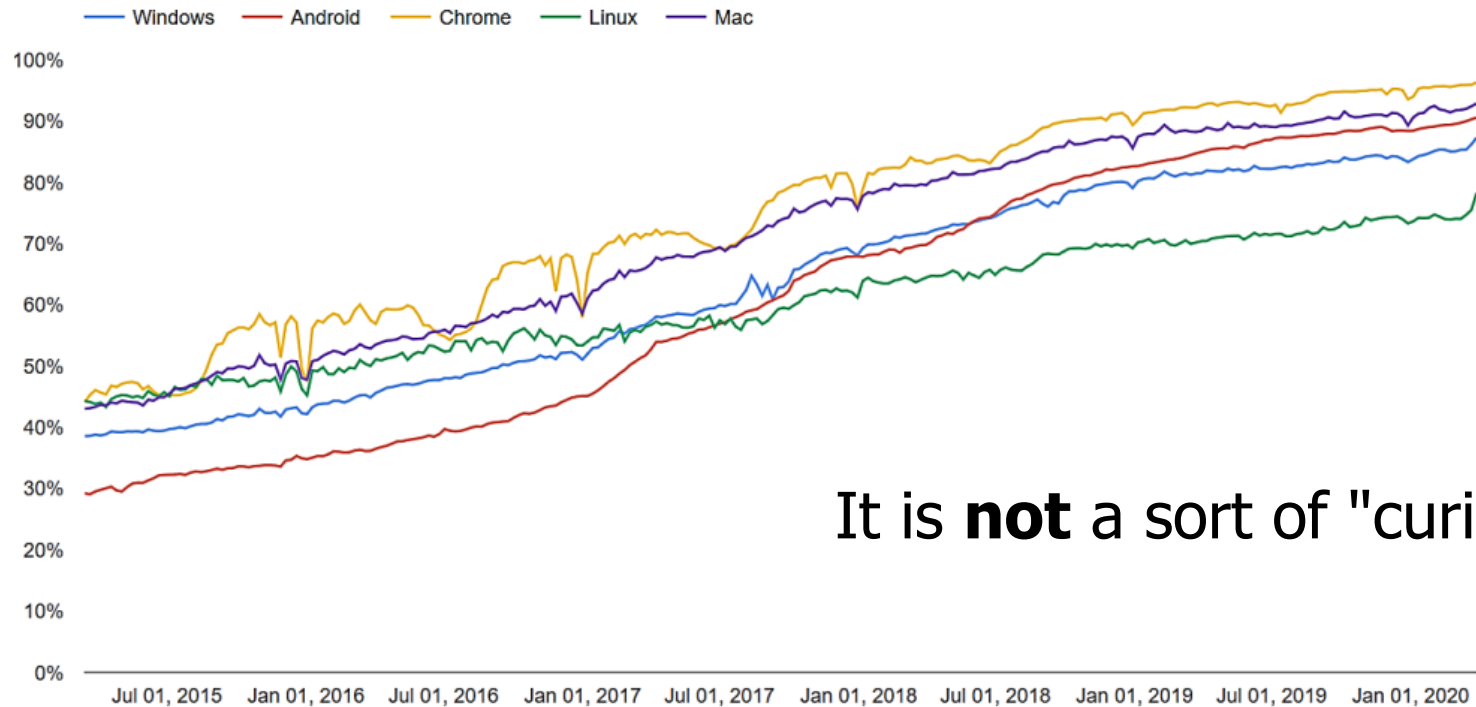
Authentication (Server DNS name)

Integrity



HTTPS adoption

Figure 5: Google report from April 2020¹⁵



It is **not** a sort of "curiosity"

Keep in mind: NOT "just for browsers"



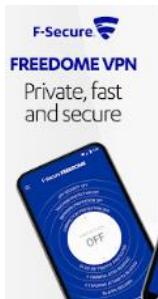
HTTPS



HTTPS



HTTPS



HTTPS



Tab: HTTP o HTTPS?

- ❑ Barra URL senza protocol
⇒ **Simbolo** rappresenta il protocol
- ❑ Browser-dependent
- ❑ Version-dependent
- ❑ Tipico per HTTPS: lucchetto chiuso

ⓘ Non sicuro | www.regione.fvg.it/rafvfg/cms/RAFVG/

🔒 <https://www.units.it>

09:18

• 4G 📶 🔋

ⓘ www.regione.fvg.it/rafvfg/cms/R

≡

⋮

09:19 ✉

• 4G 📶 🔋



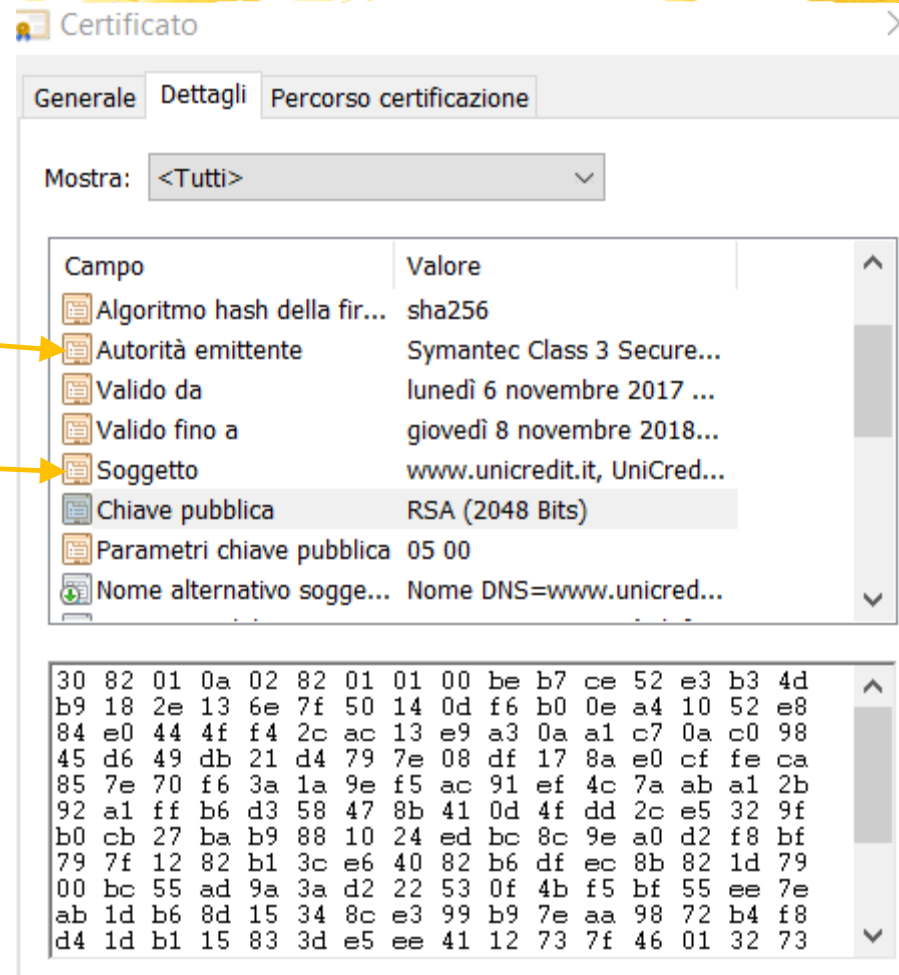
www.units.it

⋮

Esempio

Issuer

Subject

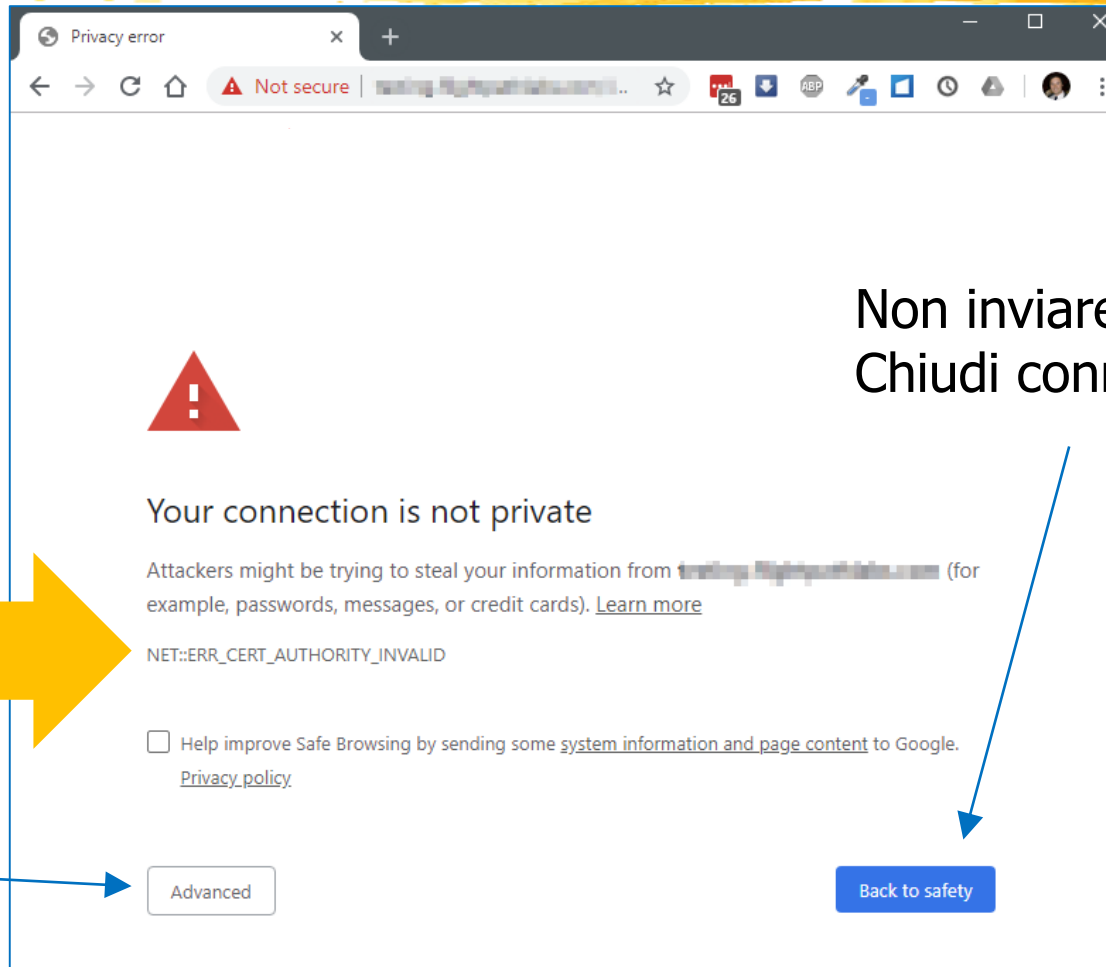


Certificato non valido: Utilizzo insicuro (REMINDE)



- ❑ SW può essere configurato per **chiedere ad utente** se usare un certificato **non valido**
 - ❑ $\text{Issuer} \notin \text{KeySet} \cap \text{TrustSet}$
oppure
 - ❑ Self-signed
- ❑ **Nessuna garanzia** sulla associazione Subject-KPUB

Esempio



Non inviare traffico HTTP
Chiudi connessione

Voglio
proseguire

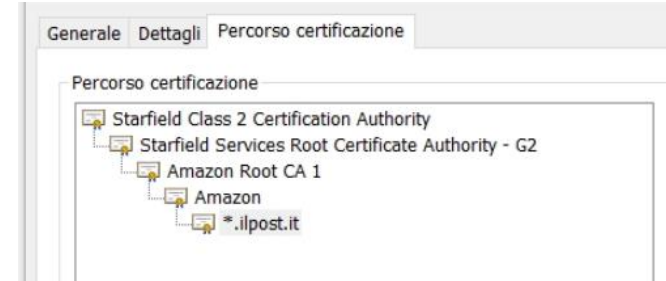
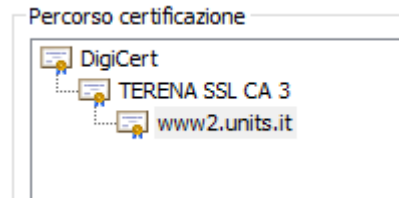
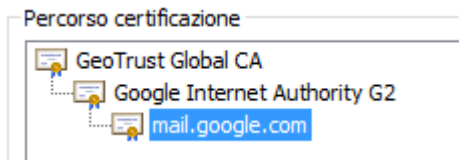
Certificato non valido:

Nota bene



- ❑ SW può essere configurato per **chiedere ad utente** se usare un certificato **non valido**
 - ❑ $\text{Issuer} \notin \text{KeySet} \cap \text{TrustSet}$
oppure
 - ❑ Self-signed
- ❑ **Nessuna garanzia** sulla associazione Subject-KPUB
- ❑ Riservatezza
- ❑ Integrità
- ~~❑ Autenticazione DNS-NAME Server~~

Certificate chain (I)

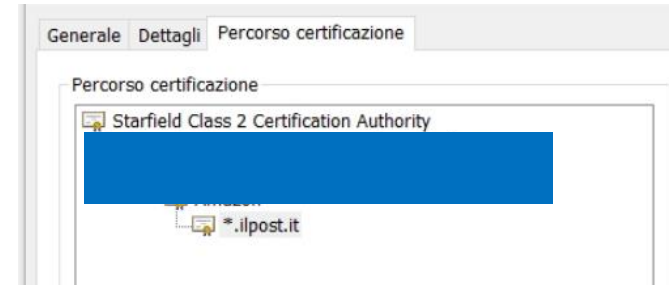


- ❑ Primo: Subject = Certification Authority
 - ❑ Deve essere contenuto nel KeySet e TrustSet
 - ❑ Anchor / Root Certificate
- ❑ Ultimo: Subject = DNS-NAME Issuer = **SomeSubject**

*Perché sono **più di due**?*

*Perché DNS-NAME è certificato da uno Issuer che **non** è la Certification Authority?*





- 📄 Spiegazioni dettagliate a Reti II

TLS: Implementazione (ultra-ultra-semplificata)

Osservazione



- ❑ Molto più semplice da quella reale
- ❑ Adeguata per i nostri scopi didattici

- ❑ 2020/2021: Aggiunto un dettaglio molto importante in pratica (semplice da capire)
- ❑ 2023/2024: Modificato l'uso di crittografia / HMAC per renderlo più realistico (esercizi svolti non modificati)
(mi accontento anche dell'uso degli anni scorsi...)

Implementazione TLS (ultra-ultra-semplificata) (I)

Client:

1. Ottiene IP-N di **DNS-NAME**
2. `connect()` con porta 443 di IP-N
- ☐ Send "**DNS-NAME**"

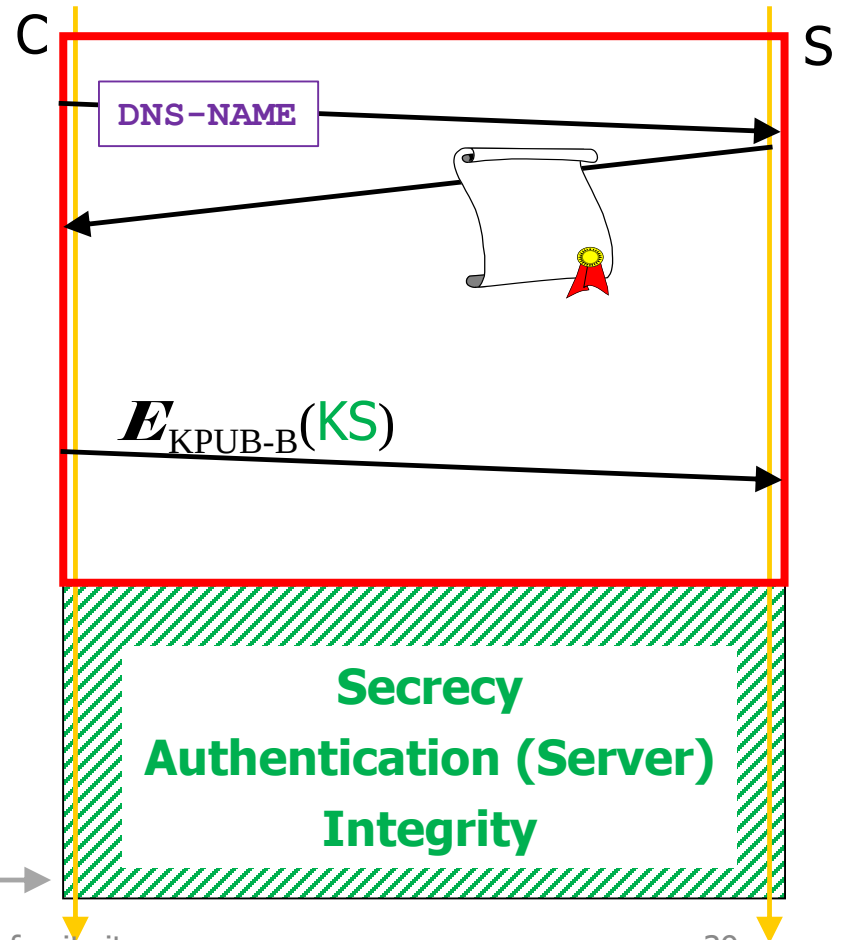
☐ Riceve certificato CERT

☐ Verifica CERT

☐ Verifica CERT.Subject = **DNS-NAME**

☐ Sceglie chiave KS

☐ Trasmette $E_{\text{CERT.KPUB}}(\text{KS})$
4. `send()/receive()` HTTP



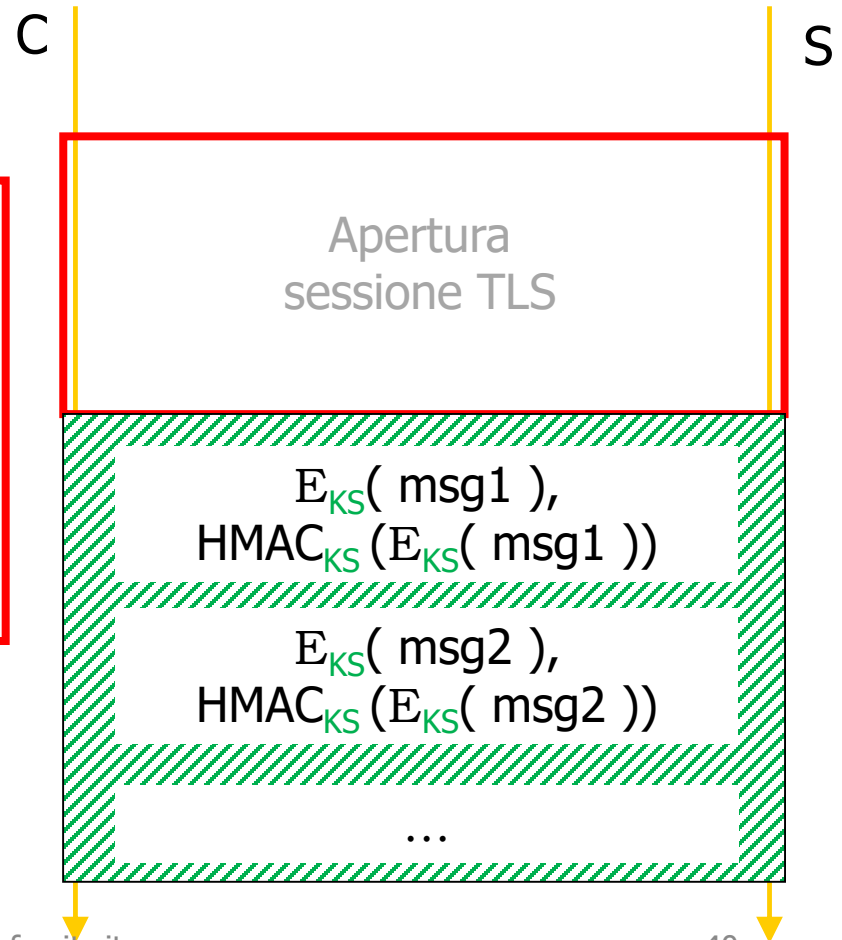
Implementazione TLS (ultra-ultra-semplificata) (II)

Client:

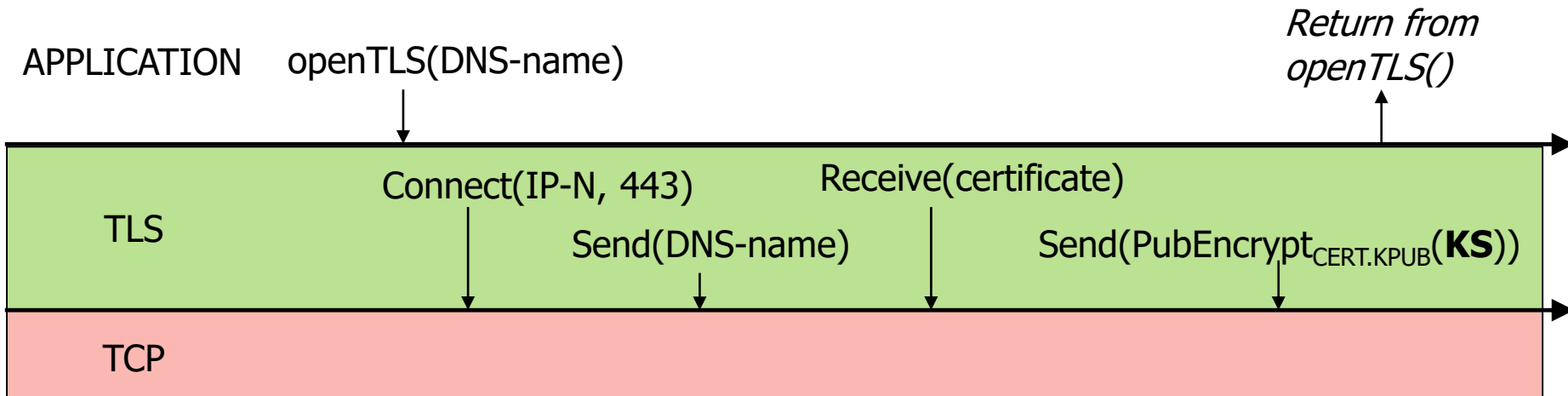
1. Ottiene IP-N di **DNS-NAME**
2. `connect()` con porta 443 di IP-N

- ☐ Send "**DNS-NAME**"
- ☐ Riceve certificato CERT
- ☐ Verifica CERT
- ☐ Verifica `CERT.Subject = DNS-NAME`
- ☐ Sceglie chiave KS
- ☐ Trasmette $E_{\text{CERT.KPUB}}(\text{KS})$

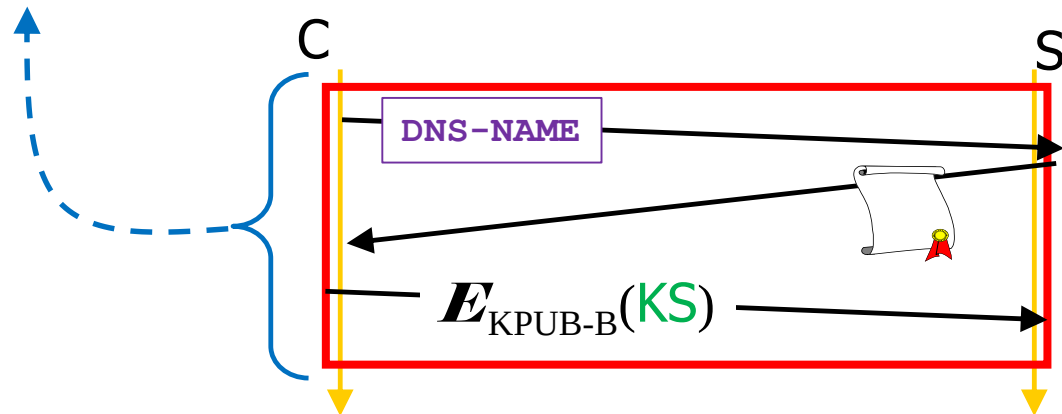
- ☐ Communicate with KS
 - ☐ Encrypt, Compute MAC, `send()`
 - ☐ `receive()`, Verify MAC, Decrypt,



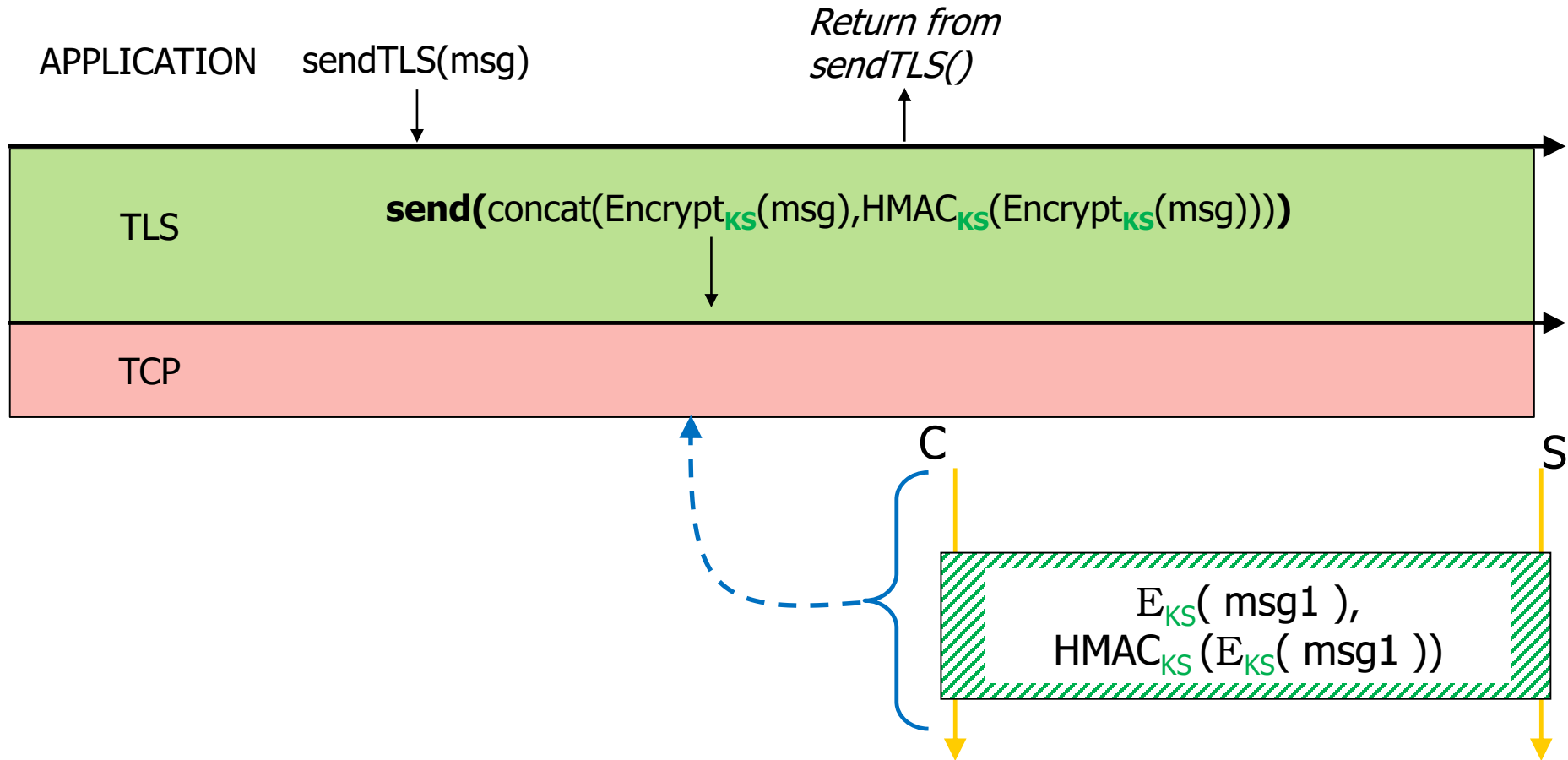
Altro punto di vista (I)



- ☐ Risoluzione DNS-name omessa
- ☐ Procedure socket semplificate (omettono il socket)



Altro punto di vista (II)



Hmmm...perché?

Client:

1. Ottiene IP-N di **DNS-NAME**
2. `connect()` con porta 443 di IP-N
- ☐ Send "**DNS-NAME**"

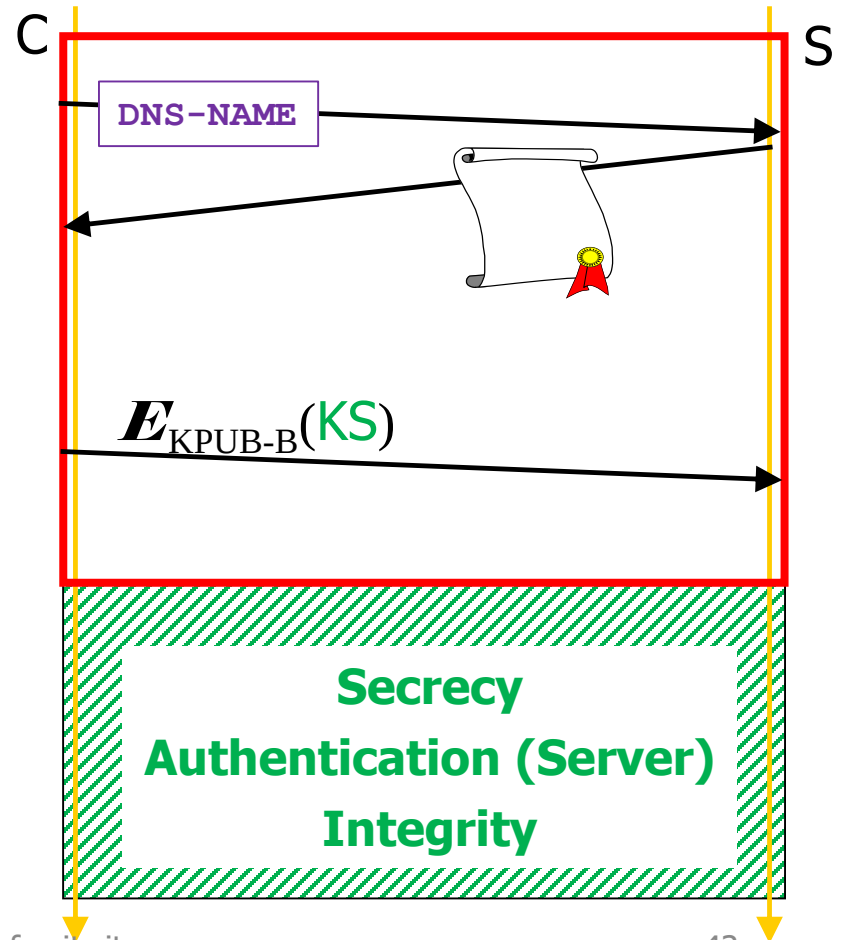
☐ Riceve certificato CERT

☐ Verifica CERT

☐ Verifica **CERT.Subject = DNS-NAME**

☐ Sceglie chiave KS

☐ Trasmette $E_{\text{CERT.KPUB}}(\text{KS})$
4. `send()/receive()` HTTP



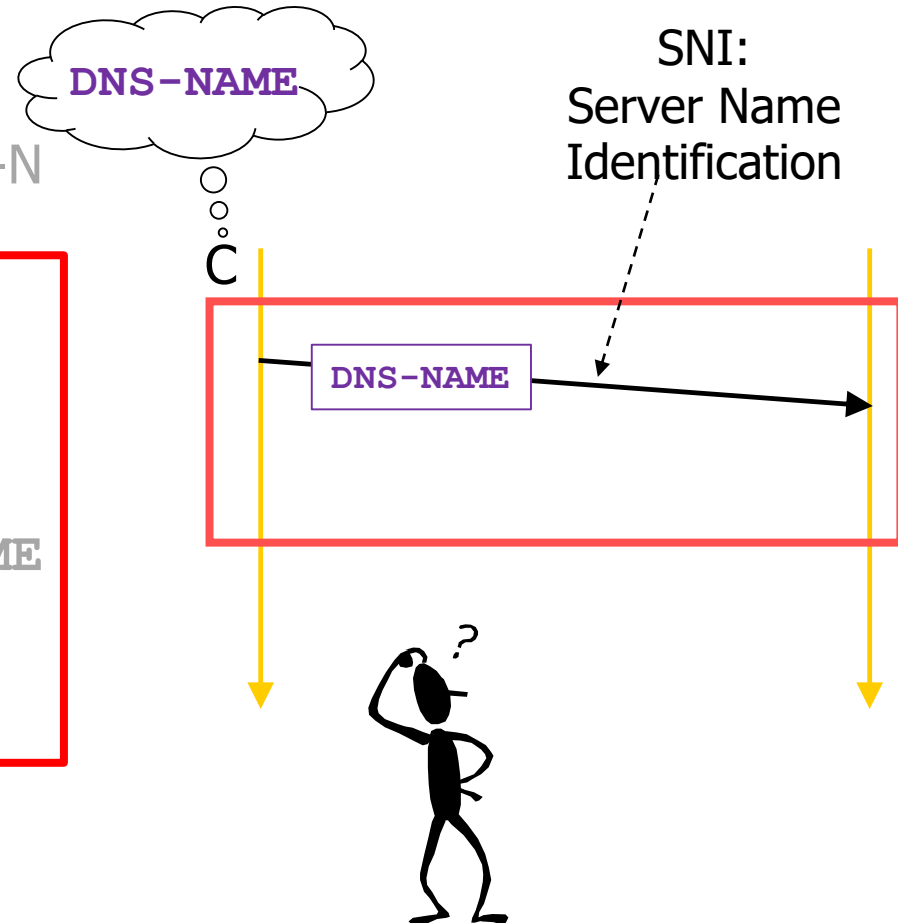
Perché ripete DNS-NAME?

Client:

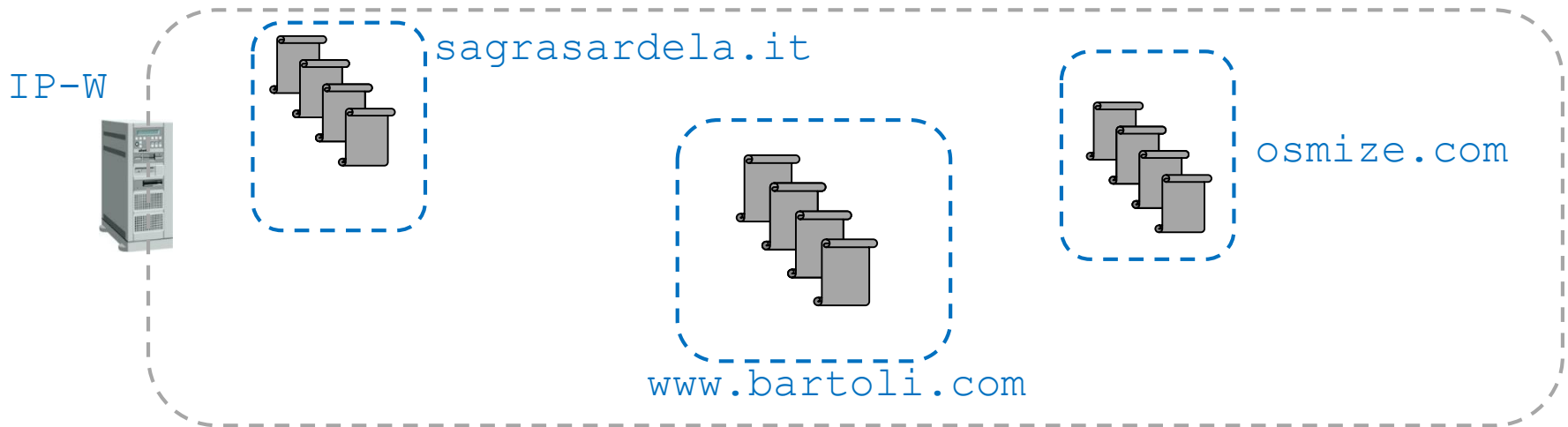
1. Ottiene IP-N di **DNS-NAME**
2. `connect()` con porta 443 di IP-N

- ☐ Send "**DNS-NAME**"
- ☐ Riceve certificato cert
- ☐ Verifica cert
- ☐ Verifica `cert.Subject = DNS-NAME`
- ☐ Sceglie chiave KS
- ☐ Trasmette $E_{\text{cert.KPUB}}(\text{KS})$

4. `send()/receive()` HTTP



Web hosting



- ❑ Un web server può ospitare molti siti web (quindi può avere **molti nomi diversi**)
⇒ Deve avere chiavi+certificato distinti per ogni nome
- ❑ SNI permette al Client di specificare il nome desiderato
 - ❑ **Non** può farlo con `Host` (il traffico HTTP non è ancora iniziato!)

Perché controlla il nome sul certificato?

Client:

1. Ottiene IP-N di **DNS-NAME**
2. `connect()` con porta 443 di IP-N

- ☐ Send "DNS-NAME"
- ☐ Riceve certificato cert
- ☐ Verifica cert
- ☐ Verifica `cert.Subject = DNS-NAME`
- ☐ Sceglie chiave KS
- ☐ Trasmette $E_{\text{cert.KPUB}}(\text{KS})$

4. `send()/receive()` HTTP



NON deve accettare qualsiasi certificato (I)

Client:

1. Ottiene IP-N di **DNS-NAME**
2. `connect()` con porta 443 di IP-N

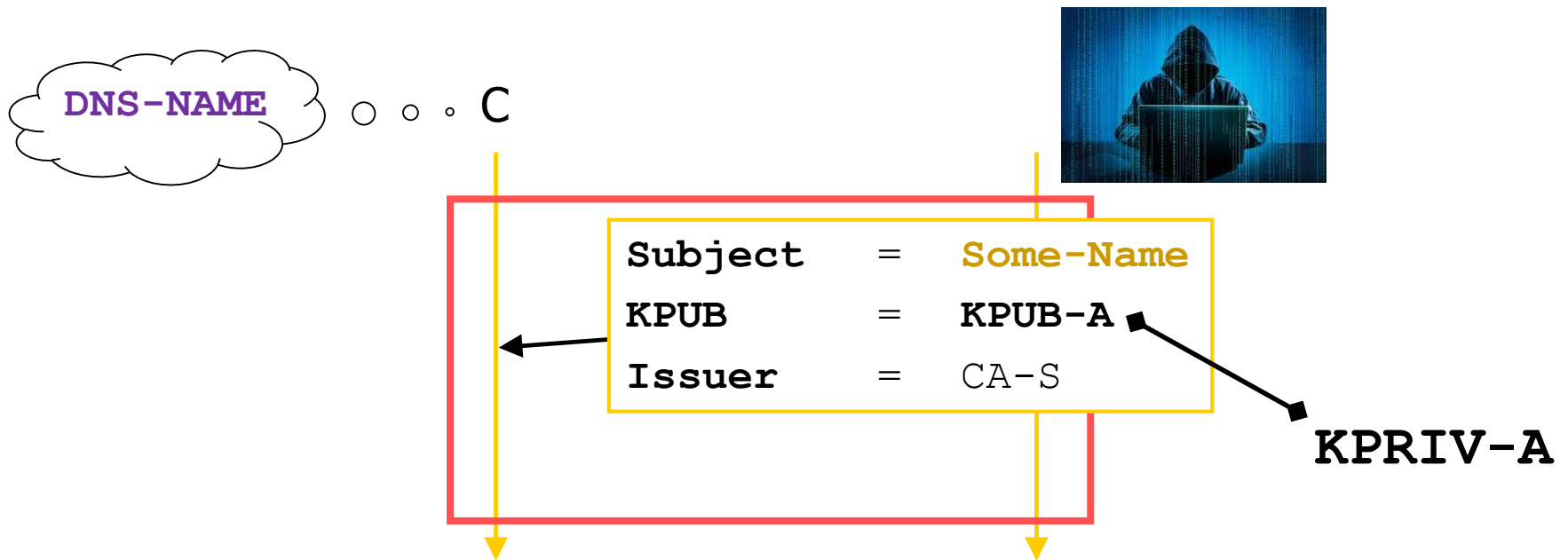
- ☐ Send "DNS-NAME"
- ☐ Riceve certificato cert
- ☐ Verifica cert
- ☒ ~~Verifica cert.Subject == DNS-NAME~~
- ☐ Sceglie chiave KS
- ☐ Trasmette $E_{\text{cert.KPUB}}(\text{KS})$

4. `send()/receive()` HTTP

Browser
accetterebbe
qualsiasi certificato

NON deve accettare qualsiasi certificato (II)

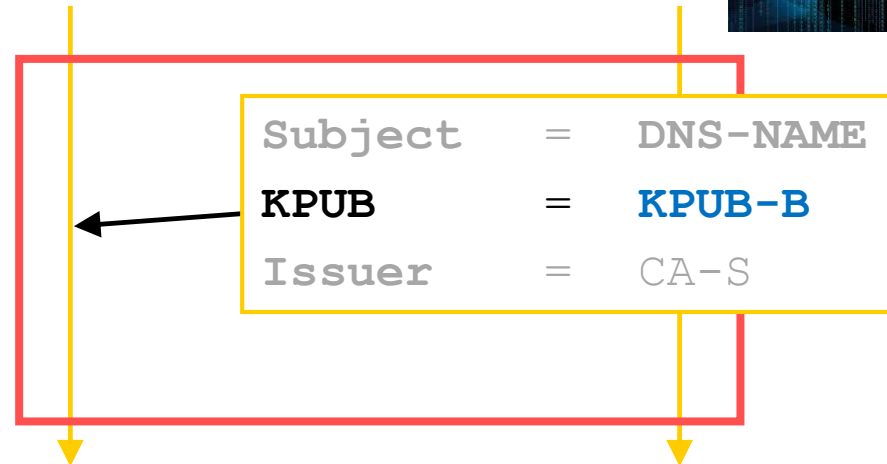
- ❑ Network Attacker si fa rilasciare certificato **legittimo** per **Some-Name**
- ❑ "Man-In-The-Middle" tra Client e **DNS-NAME**



Hhmmm...



*I certificati sono **pubblici**...
...quindi potrebbe avermelo
inviato un impostore*



No problem!



DNS-NAME

... C

- ❑ Verifica CERT
- ❑ Verifica CERT.Sub $\neq$ DNS-NAME
- ❑ Sceglie chiave KS

Subject	=	DNS-NAME
KPUB	=	KPUB-B
Issuer	=	CA-S

$E_{KPUB-B}(KS)$

NON può decrittare KS
(non ha KPRIV-B)

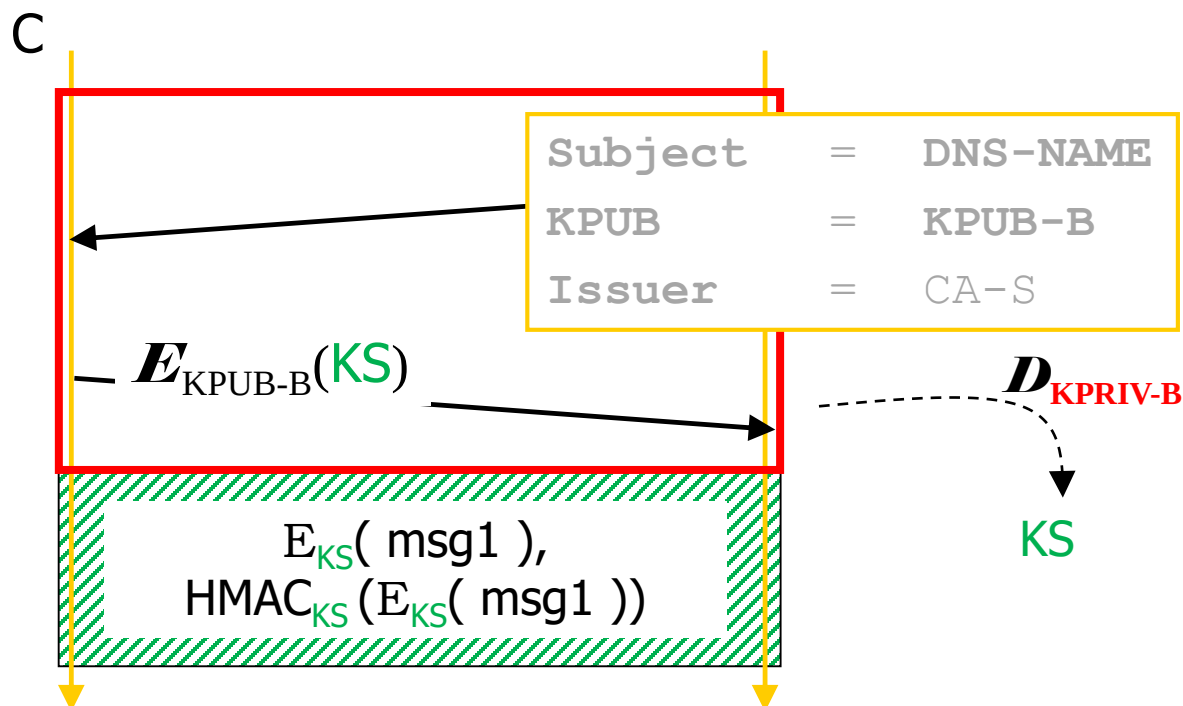
NON può generare
HMAC_{KS} corretto



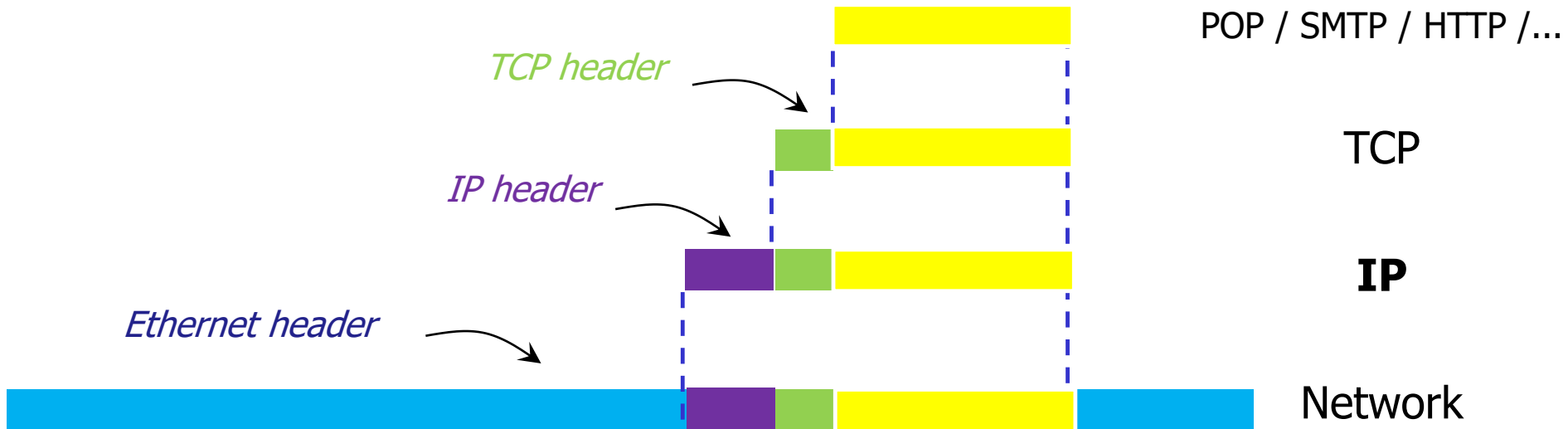
Ecco come dimostra di conoscere il segreto...!

- DNS-NAME dimostra di conoscere **un segreto**
- ...senza che lo conosca nessun altro (!!!)

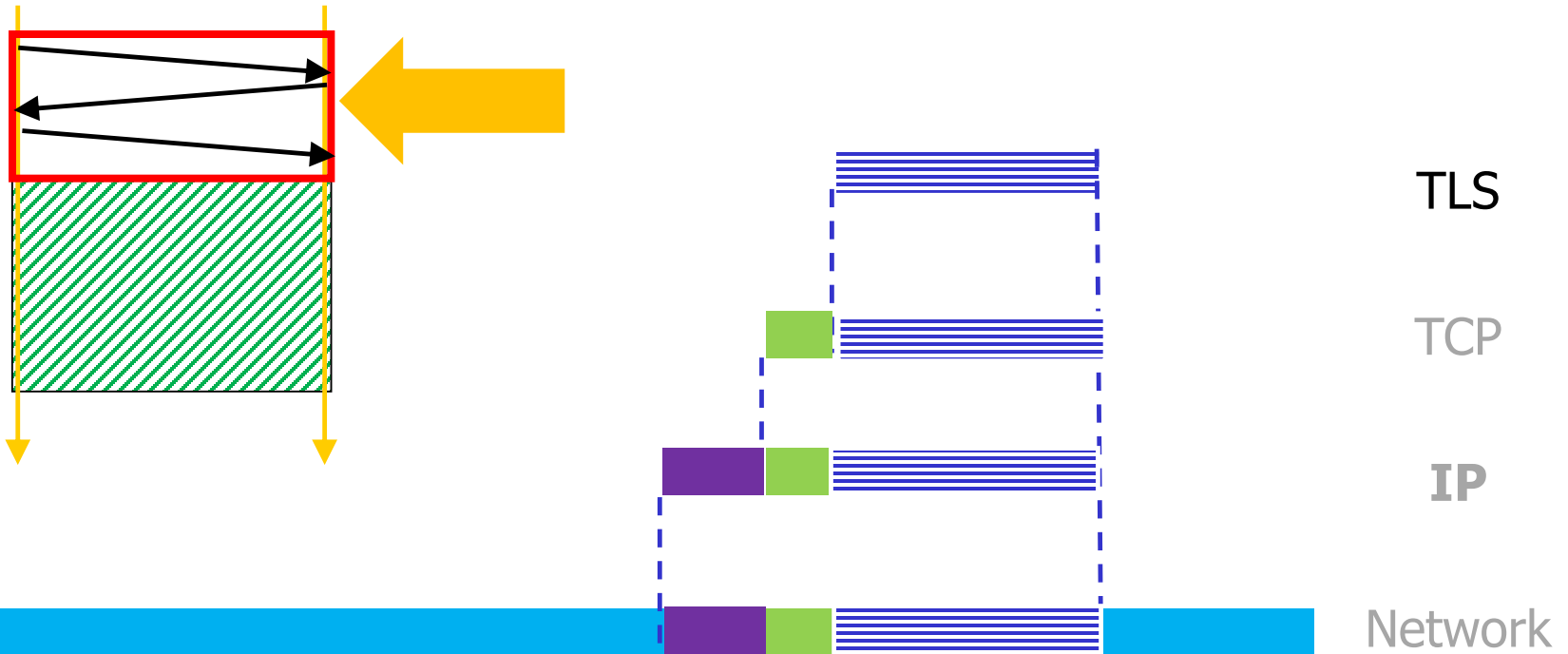
Dimostra di sapere decrittare KS!



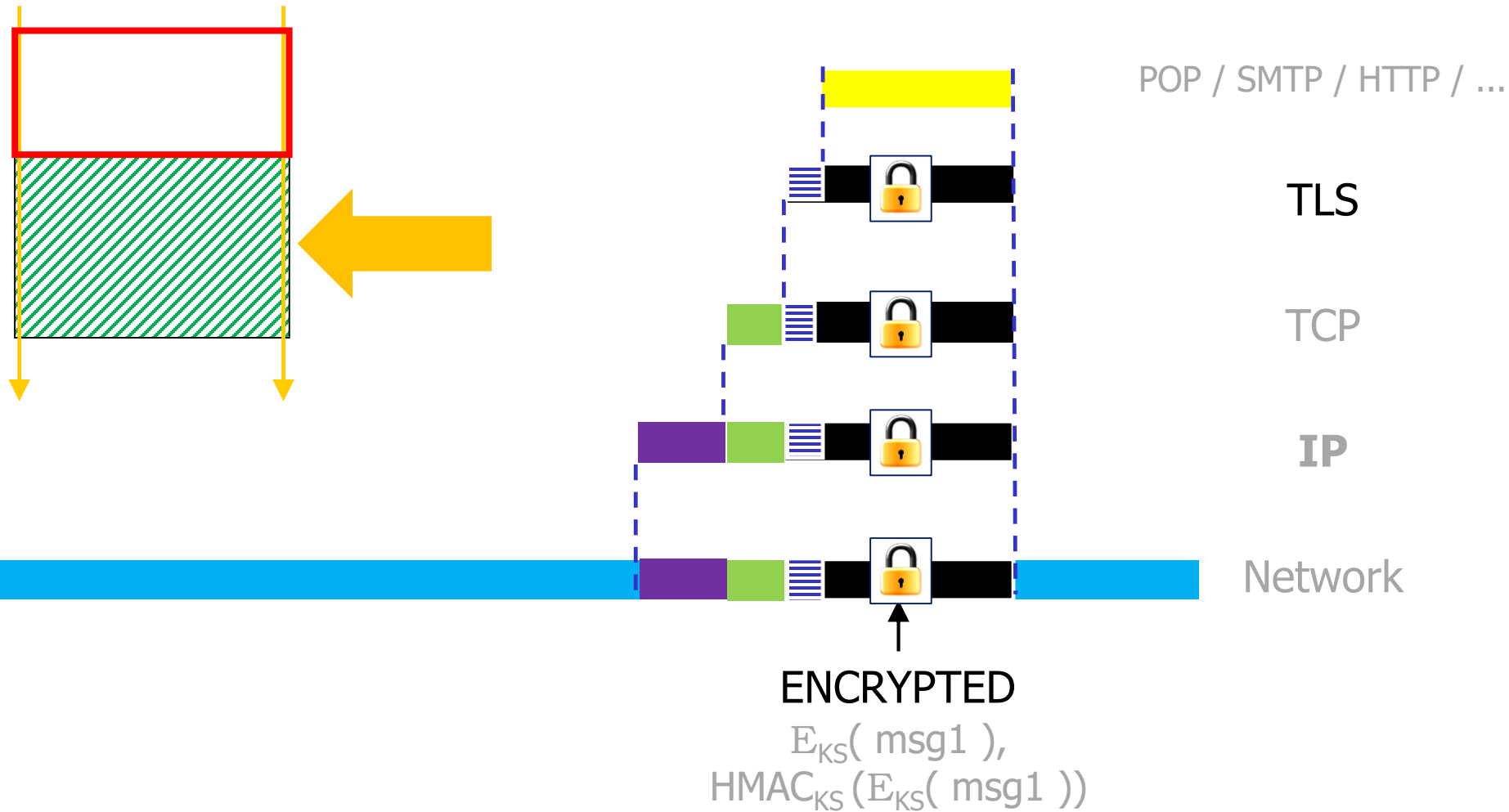
Incapsulamento: Senza TLS



Incapsulamento: Con TLS (I)

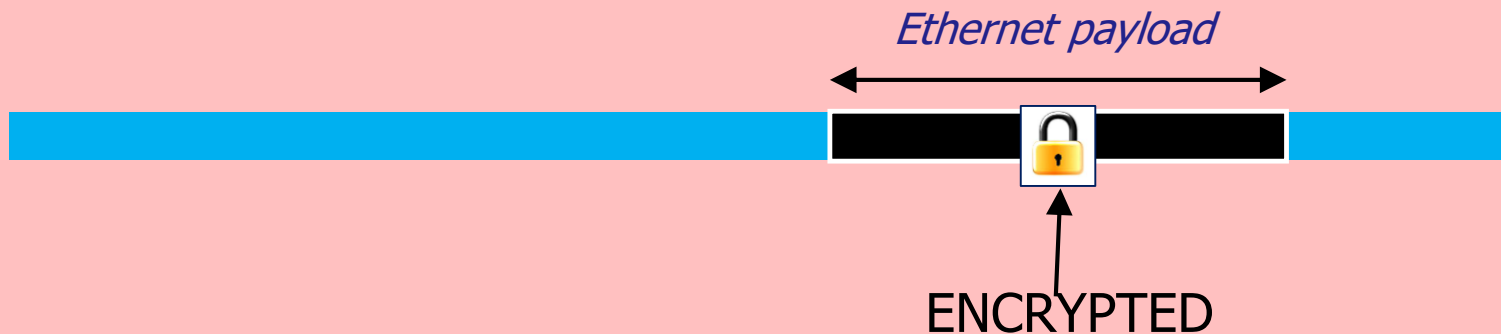


Incapsulamento: Con TLS (II)



Bocciatura automatica

...Indicare quali porzioni del pacchetto e del frame sono crittate...



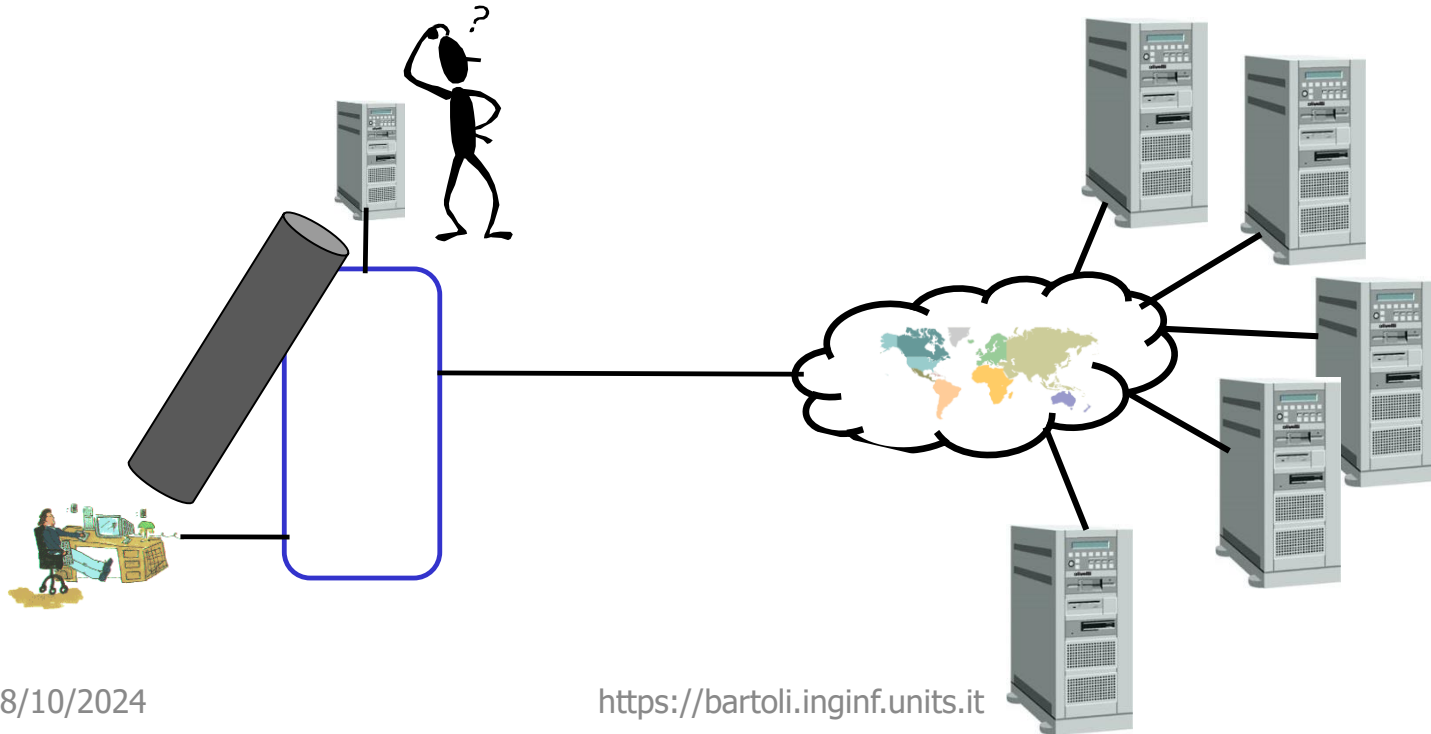
Se è **crittato il pacchetto** come caspita possono instradarlo i router???

Qual'è IP-DST?

Mistero...

Come fa a funzionare attraverso i **proxy** ?

*TLS è su **una** connessione TCP...qui ce ne devono essere **due***

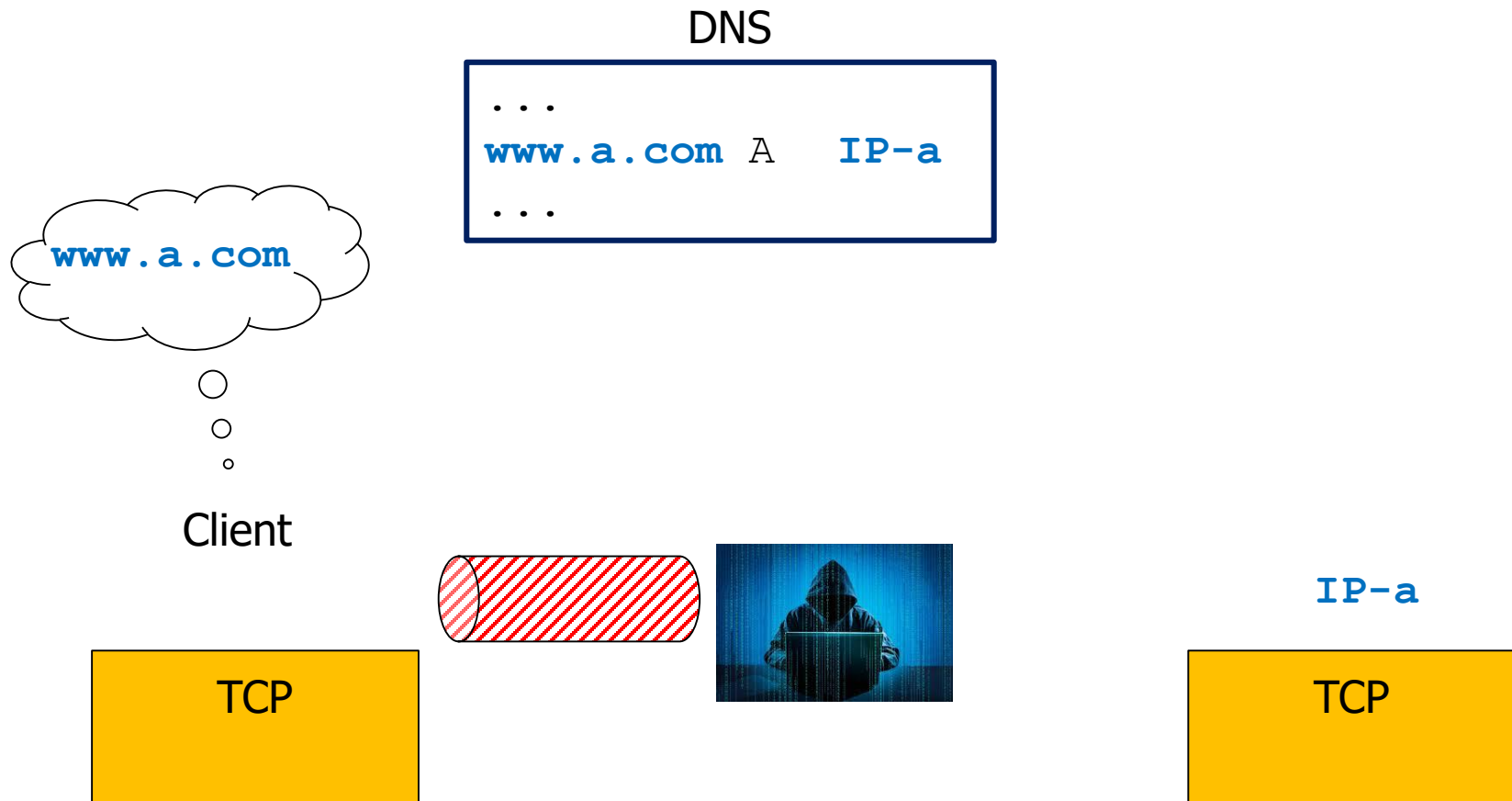


Attacker:

Esempi chiave



DNS spoofing



Attacker: Casi significativi

A

Subject	=	www.someother.com
KPUB	=	KPUB-ATTACKER
Issuer	=	CA-S

B

Subject	=	www.a.com
KPUB	=	KPUB-ATTACKER
Issuer	=	CA-S

C1

Subject	=	www.a.com
KPUB	=	KPUB-ATTACKER
Issuer	=	CA-S'

← ∈ Trust-Set di C

C2

Subject	=	www.a.com
KPUB	=	KPUB-ATTACKER
Issuer	=	CA-ATTACKER

← ∉ Trust-Set di C

Caso A:

Client si rifiuta

Client:

1. Ottiene IP-N di **www.a.com**
2. `connect()` con porta 443 di IP

Subject	=	www.someother.com
KPUB	=	KPUB-ATTACKER
Issuer	=	CA-S

- ☐ Send "**www.a.com**"
- ☐ Riceve certificato cert
- ☐ Verifica cert
- ☐ Verifica cert.Subject = **www.a.com**
- ☐ Sceglie chiave KS
- ☐ Trasmette $E_{\text{cert.KPUB}}(\text{KS})$

4. `send()/receive()` HTTP



Caso B:

Non può accadere

Subject	=	www.a.com
KPUB	=	KPUB-ATTACKER
Issuer	=	CA-S



- ❑ Ogni associazione contenuta in un certificato è **vera**
- ❑ *"Ogni CA in TrustSet dice sempre la verità"*

Caso C1:

Non può accadere

Subject	=	www.a.com
KPUB	=	KPUB-ATTACKER
Issuer	=	CA-S'



←

↑

←

∈ Trust-Set di C

- ❑ Ogni associazione contenuta in un certificato è **vera**
- ❑ *"Ogni CA in TrustSet dice sempre la verità"*

Caso C2:

Certificato non valido

Subject	=	www.a.com
KPUB	=	KPUB-ATTACKER
Issuer	=	CA-ATTACKER



- ❑ CA-Attacker non è in TrustSet
- ❑ Verifica certificato fallisce

Server authentication: In parole povere

"The server name is **authentic**"

□ IF

the connection with **DNS-NAME** is established

□ THEN

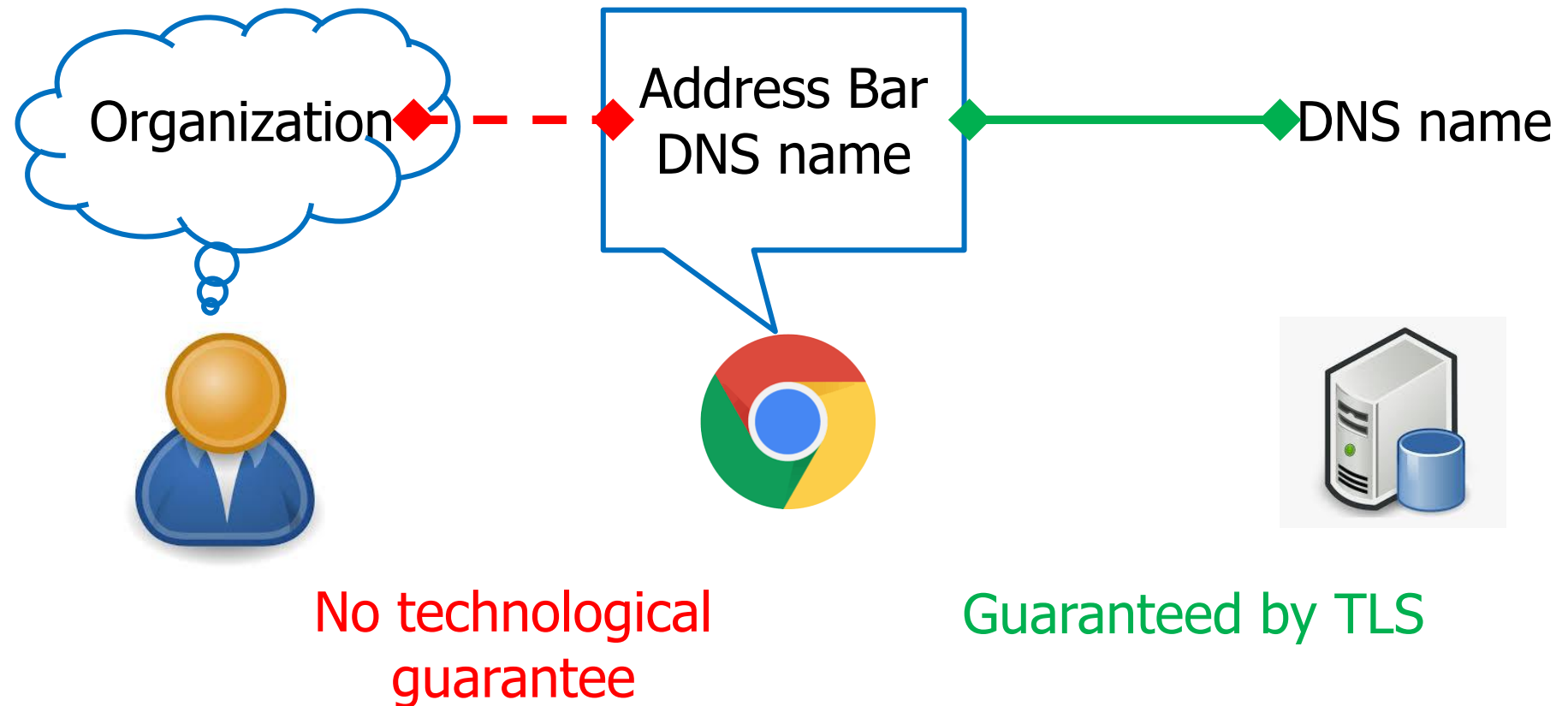
At the other end there is the server
with IP address associated with **DNS-NAME**

Address Bar
DNS name

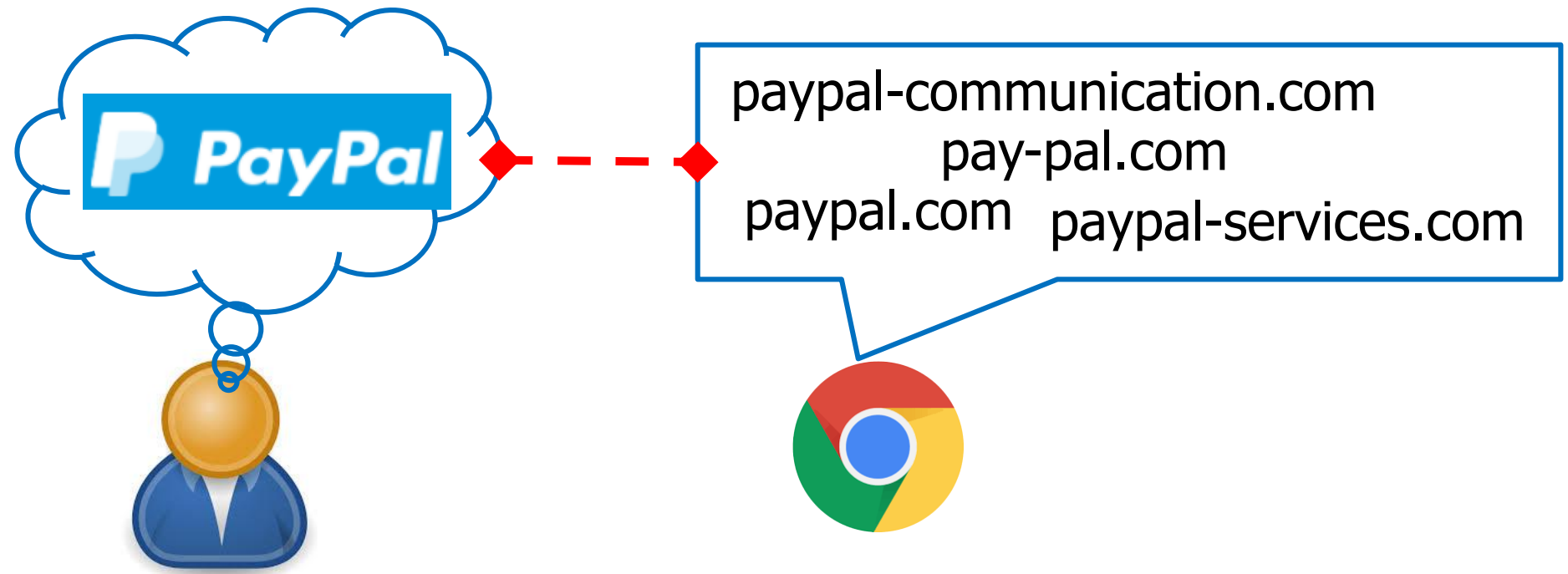


DNS-NAME

Extremely important



Example 1



No technological
guarantee

Example 2

idp-cineca-units.it	✓	6,99 €/year
idp-cineca-units.eu	✓	6,99 €/year

https://idp-cineca.units.it/idp/profile/SAML2/Redirect/SSO?execution=e1s2

Welcome

WELCOME, The University of Trieste has implemented authentication with SPID (Public)
As an alternative to SPID credentials, you can continue to use the university credentials

 **UNIVERSITÀ
DEGLI STUDI DI TRIESTE**

Sign in to the service **sp-esse3-units-prod.cineca.it**

Username

Password

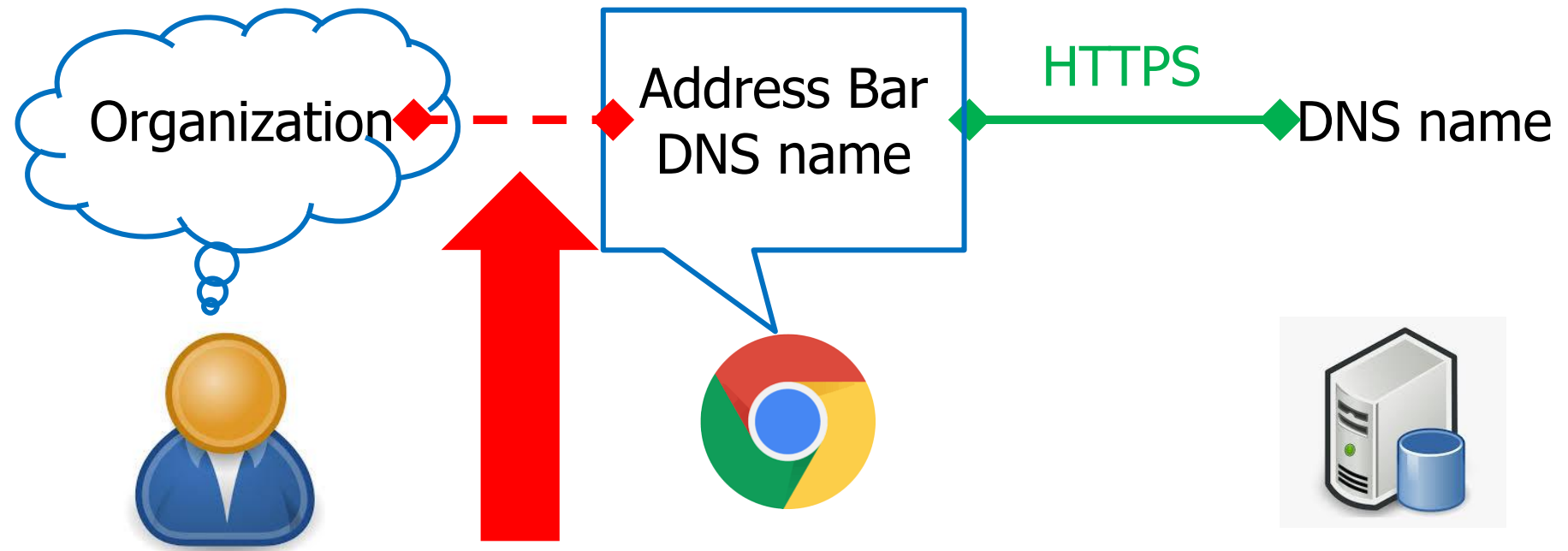
Login

 **Entra con SPID**

esse3-units.it	✓
esse3-units.eu	✓

esami-unitrieste.it	✓
esami-unitrieste.eu	✓

Keep in mind



Key attack point
Now and in the future

Uso di HTTPS: Capiamolo meglio



Secrecy? (I)

Posso visitare Wikipedia
senza che un
network attacker se ne accorga

E' proprio così?



Client

Server



Secrecy? (II)

Posso visitare la pagina P-X di Wikipedia
senza che un
network attacker se ne accorga

E' proprio così?



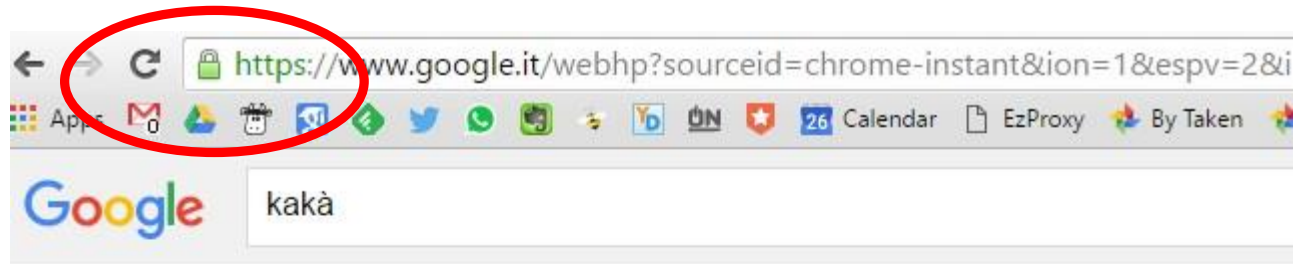
Client



Server

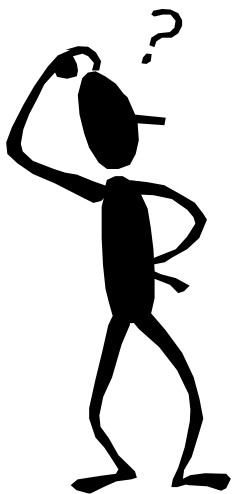


A che serve quando non mi autentico?



Perché su HTTPS?

*Non ho dovuto inserire
password...*



All Images Videos News Maps More ▾ Search tools

About 541,000 results (0.51 seconds)

Kaká - Wikipedia

<https://it.wikipedia.org/wiki/Kaká> ▾ Translate this page

Ricardo Izecson dos Santos Leite, meglio noto come **Kaká** (hi'kardu i'zeksõn dus_ 'sẽrtus 'leĩtĩ; Gama, 22 aprile 1982), è un calciatore brasiliano, ...

[Biografia](#) - [Caratteristiche tecniche](#) - [Carriera](#) - [Statistiche](#)

Kaká - Wikipedia, the free encyclopedia

<https://en.wikipedia.org/wiki/Kaká> ▾

Ricardo Izecson dos Santos Leite commonly known as **Kaká** or Ricardo **Kaká**, is a Brazilian professional footballer who plays as an attacking midfielder for ...

[Orlando City SC](#) - [Caroline Celico](#) - [São Paulo FC](#) - [Digão \(footballer, born 1985\)](#)

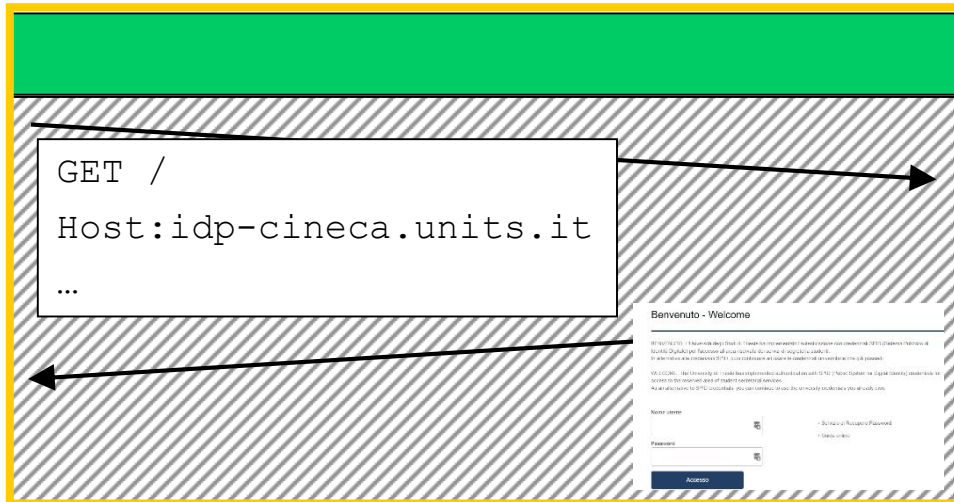
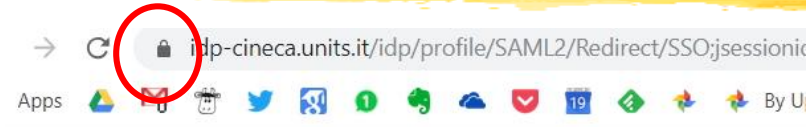
S+A+I



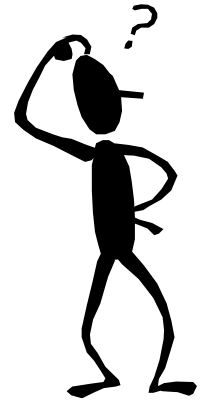
- ❑ Garantisce:
 - ❑ **Secrecy**
Ricerca e risultati visibili solo a me e a Google
 - ❑ **Authentication**
Risultati ricerca generati veramente **da Google**
 - ❑ **Integrity**
Risultati ricerca **non alterati** in transito

- ❑ Fantastico: NetworkAttacker non può portarmi dove vuole lui

A che serve PRIMA di inviare credenziali?



- Ma a che serve prelevare un form con HTTPS?



- Non ho ancora inserito password... non mi interessa che il prelievo del form sia **riservato**

S+A+I



- ❑ Che sia **riservato** non importa...
- ❑ ...ma è **fondamentale** che sia **autentico** e **integro** !

- ❑ FORM su HTTPS
- ❑ NetworkAttacker può:
 - ❑ Impersonare Server

 - ❑ Modificare ACTION del FORM
 - ❑ Aggiungere keylogger Javascript (1 linea della risposta)
 - ❑ ...

"Rubare il cookie" a siti HTTPS (???)



HTTPS = Cookie protection



- ❑ Scenario:
 - ❑ Victim authenticated on webapp W
 - ❑ W maintains authenticated sessions on **HTTPS**

- ❑ Threat model:
 - ❑ Network Attacker **Observe**

- ❑ Attacker **cannot** grab cookies of W

INTRINSIC Vulnerability



- ❑ Scenario:
 - ❑ Victim authenticated on webapp W
 - ❑ W maintains authenticated sessions on **HTTPS**

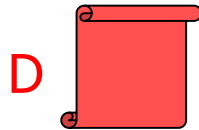
- ❑ Threat model:
 - ❑ Network Attacker **Observe**
 - +
 - ❑ Victim visits Attacker-controlled URL **while logged on W**

- ❑ **Attacker grabs authentication cookie of W (!!!)**

Malicious web page



W

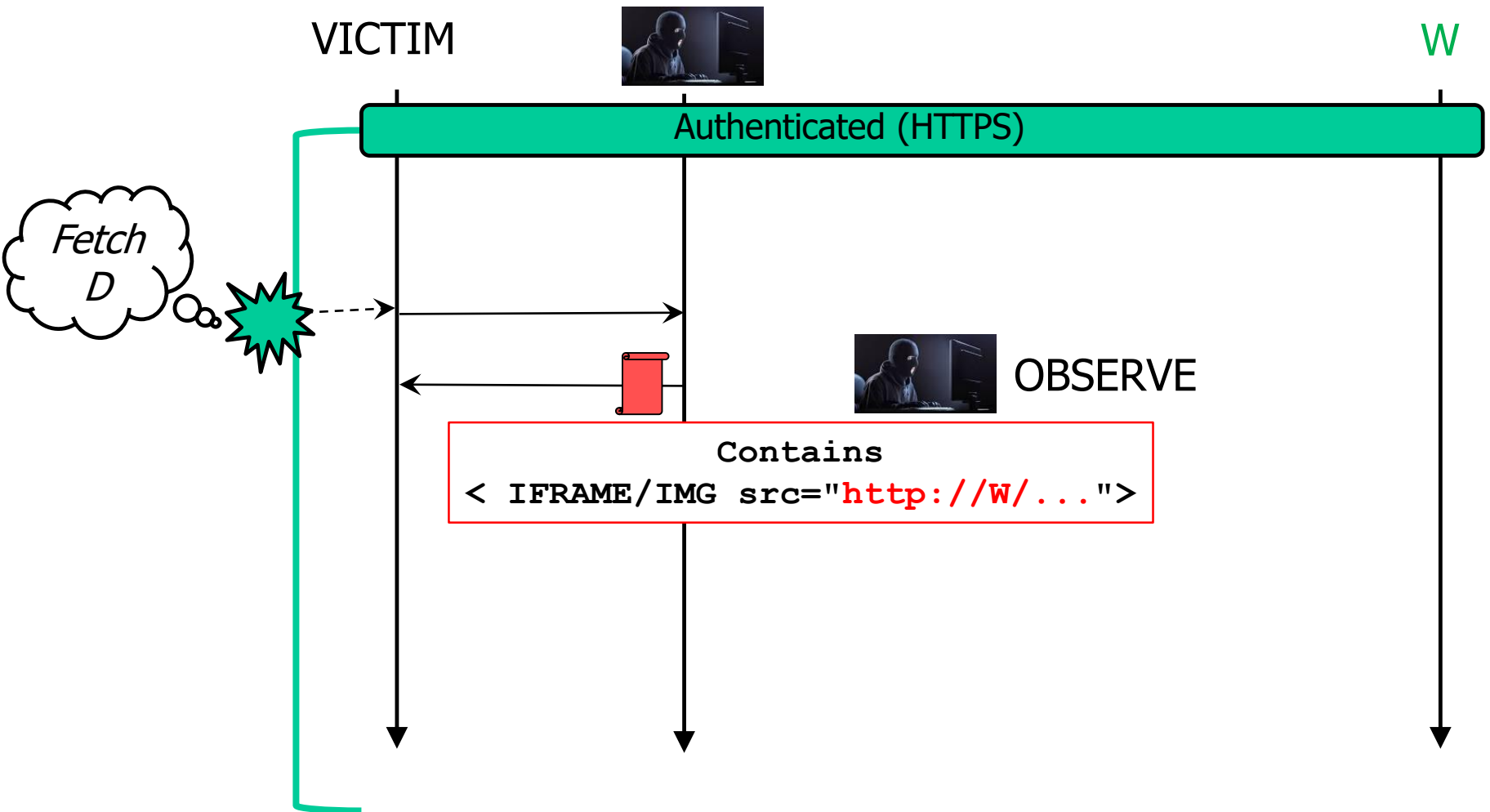


Contains

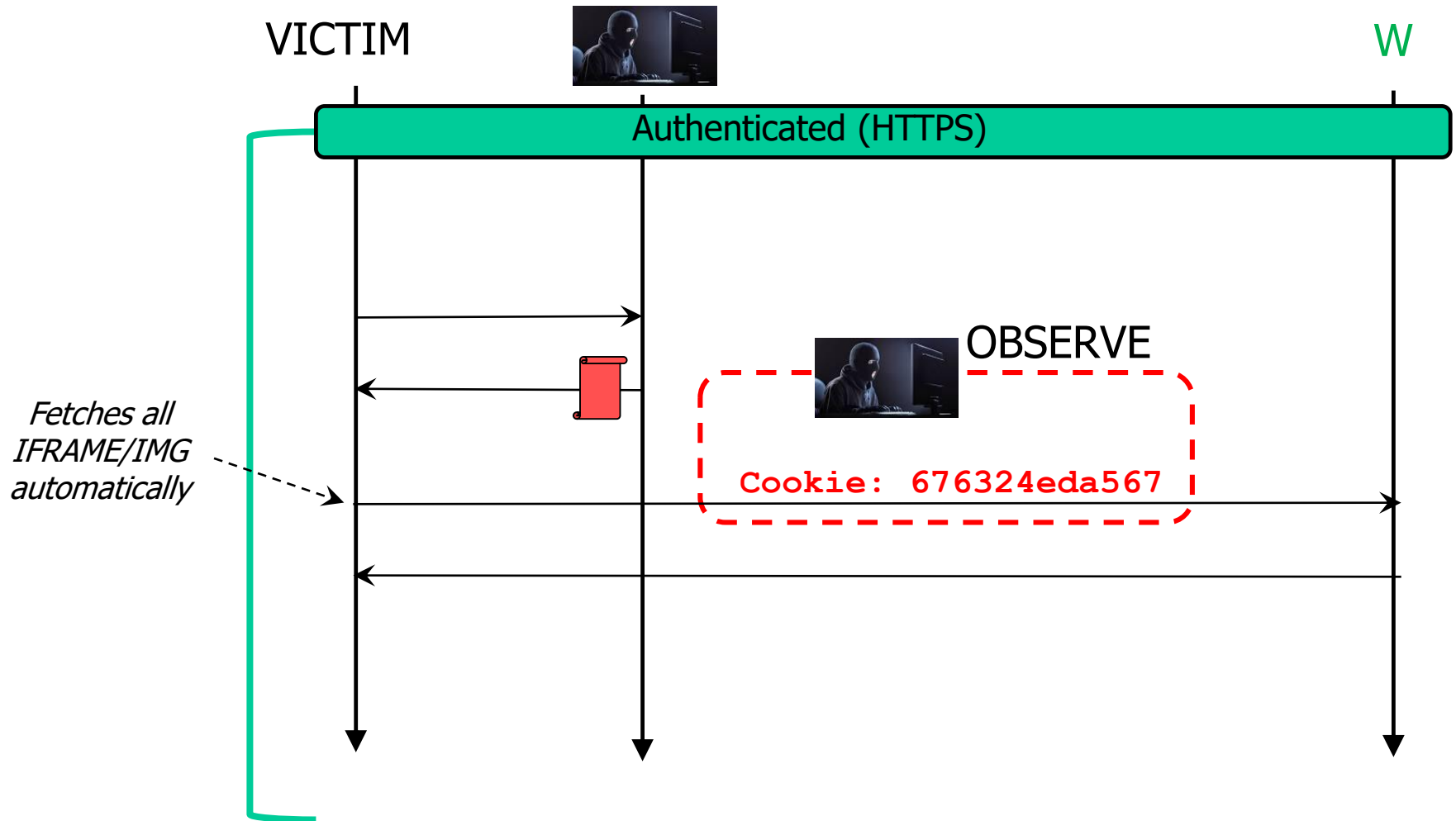
```
< IFRAME/IMG src="http://W/...">
```



Example (I)



Example (II)



General defense:

Secure Cookie

- ❑ Webapp W maintains authenticated sessions on **HTTPS**
- ❑ W must configure cookies with the `Secure` attribute



- ❑ Browser will **never** send a `Secure` cookie on HTTP
- ❑ Browser visits Attacker-controlled URL
- ❑ Attacker-controlled URL contains HTTP link to W
 - ❑ Browser will send request to W **without** `Secure` cookie

Example (some time ago)

https://esse3.units.it/auth/docente/AreaDocente.do

- .units.it | _ga
- .units.it | _gid
- .units.it | amplitude_id_408774472b1245a7df5814f20e7484d0units.it
- .units.it | unitscookieconsent
- esse3.units.it | _shibsession_6573736533756e69747368747470733a2f2f73702d6573736533
- esse3.units.it | accepted_cookie_policy
- esse3.units.it | JSESSIONID

Valore
FB426CA6F02AB425589A1A599191DD08.esse3-units-prod-04

Dominio
esse3.units.it

Percorso
/

Scadenza
Thu Apr 02 2020 12:21:33 GMT+0200 (Central European Summer Time)

SameSite
No Restriction

Solo Host ☒ Sessione ☒ Sicuro ☒ Solo Http ☒

✓ Aiuto

https://www.telepass.com/KTI/dashboard

- .telepass.com | _ga
- .telepass.com | _gat_UA-89348357-1
- .telepass.com | _gcl_au
- .telepass.com | _gid
- www.telepass.com | JSESSIONID

Valore
0000XtDDvvG_6-PonVr_ZBTNBAN:1apnk2uup

Dominio
www.telepass.com

Percorso
/

Scadenza
Thu Apr 02 2020 12:22:40 GMT+0200 (Central European Summer Time)

SameSite
No Restriction

Solo Host ☒ Sessione ☒ Sicuro ☐ Solo Http ☒

✓ Aiuto

TEST - HTTPS

A thick, horizontal yellow brushstroke with a textured, painterly appearance, spanning most of the width of the slide.